

HPC 优化工程篇：构建、Profiling 与方法论

目 录

1 Part I: CPU 工程实践	5
1.1 构建与 Baseline 检视	5
1.2 引言：从原理到实践	5
1.3 获取代码与构建	5
1.3.1 克隆仓库	5
1.3.2 CMake 构建配置	6
1.3.3 构建与运行	6
1.4 运行 Baseline：建立性能基线	6
1.5 读代码：理解 Baseline 的访存模式	7
1.5.1 逐 token 遍历的结构	7
1.5.2 expert_ffn 的访存模式	8
1.6 自动向量化：编译器做了什么	8
1.6.1 诊断解读	9
1.6.2 为什么“16 byte vectors”	9
1.7 汇编检视：objdump	9
1.7.1 汇编解读	10
1.8 Compiler Explorer：在线对比	10
1.8.1 基本使用流程	10
1.8.2 对比不同编译选项	11
1.9 环境差异说明	11
1.10 数据布局与 Preprocess	11
1.11 引言：为什么数据布局决定了优化上限	11
1.12 Baseline 的访存模式	12
1.12.1 逐 token 遍历的结构	12
1.12.2 expert_ffn 的按行访问	12
1.13 preprocess：你的磨刀石	12
1.14 按专家分组：从逐 token 到逐专家	13
1.14.1 原始结构（逐 token）	13
1.14.2 分组结构（逐专家）	13
1.14.3 实现要点	14
1.15 权重布局优化	14
1.15.1 对齐分配	15
1.15.2 权重预偏移	15
1.15.3 转置 down 投影权重	15

1.16 缓存层级感知	15
1.17 分组 + VNNI 的完整框架	16
1.18 VTune 采样与 GUI 分析	17
1.19 引言：为什么要用 Profiler	17
1.20 VTune 分析类型概览	17
1.21 集群上的命令行采样	17
1.21.1 环境准备	17
1.21.2 Hotspots 采样	18
1.21.3 Microarchitecture Exploration 采样	18
1.21.4 采样注意事项	18
1.22 下载结果到本地	18
1.23 本地 GUI 分析	19
1.23.1 打开结果	19
1.23.2 Binary/Symbol Search	19
1.23.3 Bottom-up 视图	19
1.23.4 源码视图	19
1.23.5 汇编视图	20
1.24 Hotspots 分析实战	20
1.24.1 Baseline 的 Hotspots	20
1.24.2 优化版本的 Hotspots	20
1.24.3 对比分析	20
1.25 优化迭代方法论	20
1.26 引言：把工具箱串成迭代闭环	20
1.27 优化迭代方法论	21
1.27.1 四步循环	21
1.27.2 每次改动记录什么	21
1.28 推荐优化路径	21
1.28.1 Step 0: Baseline（已完成）	21
1.28.2 Step 1: 按专家分组	21
1.28.3 Step 2: VNNI 方案 A（int16 扩展）	22
1.28.4 Step 3: VNNI 方案 B（偏移技巧）	22
1.28.5 Step 4: AMX	22
1.28.6 Step 5: 多线程（可选）	22
1.29 迭代日志模板	22
1.30 性能上限分析	23
1.31 常见陷阱与解决方案	24
1.32 填充 lab2.typ 报告	24
2 Part II: GPU 工程实践	25
2.1 TileLang 语法与编程模型	25
2.2 引言：写 CUDA 太繁琐	25
2.3 TileLang 是什么	25
2.4 编程模型：tile、block、内存层次	26

2.4.1 @T.prim_func 与 thread block 编程模型	26
2.4.2 tile 抽象与内存层次	26
2.5 核心语法速览	26
2.5.1 内存分配	27
2.5.2 数据搬运	27
2.5.3 矩阵乘	27
2.5.4 循环与流水线	27
2.6 最小 GEMM 示例	27
2.7 与 CUDA/Triton 对比	29
2.8 GDN 接口与评测方式	29
2.9 引言：理解接口和数据流是优化的前提	29
2.10 函数接口详解	29
2.11 门控前缀和的 log 空间约定	30
2.12 两种输入变种	31
2.12.1 非零 initial_state	31
2.12.2 GVA: Group-Value Attention	31
2.13 计算阶段分解	31
2.14 Nsight Profiling	32
2.15 引言：优化不能靠猜	32
2.16 Profiling 的两个层次	33
2.16.1 系统级：Nsight Systems	33
2.16.2 Kernel 级：Nsight Compute	33
2.17 Nsight Systems 详解	34
2.17.1 时间线视图能告诉你什么	34
2.17.2 在 GDN 中怎么看	34
2.18 Nsight Compute 详解	34
2.18.1 关键指标解读	34
2.18.2 在 GDN 中怎么用	35
2.19 Roofline 模型：判断瓶颈类型	35
2.19.1 算术强度	35
2.19.2 Roofline 公式	36
2.20 采样与追踪	36
2.21 优化闭环	37
2.22 GDN 优化迭代方法论	37
2.23 引言：优化是一个迭代闭环	37
2.24 优化迭代方法论	37
2.25 推荐优化路径	38
2.25.1 第一步：Baseline	38
2.25.2 第二步：等价数学变换	38
2.25.3 第三步：Kernel Fusion	38
2.25.4 第四步：Shared Memory	39
2.25.5 第五步：ping-pong buffer	39

2.25.6 第六步: Warp Specialization	39
2.26 每步的预期收益与风险	39
2.27 迭代日志模板	40
2.28 性能上限分析	40
2.29 与开源实现对比	41
2.30 填写 lab3.typ 报告	41
3 附录: RISC-V 替代路径	41
3.1 引言: 从 x86 到 RISC-V	41
3.2 RISC-V 指令集基础	42
3.2.1 什么是 RISC-V	42
3.2.2 RISC 与 CISC 的区别	42
3.3 RVV 向量扩展	43
3.3.1 为什么 RVV 不一样	43
3.3.2 硬件参数: ELEN 与 VLEN	43
3.3.3 运行参数: SEW 与 LMUL	43
3.3.4 Intrinsic 编程	44
3.3.5 数据类型	45
3.3.6 vsetvl 系列指令	45
3.3.7 常见操作速查	45
3.3.8 Self Check: 内存中的数据有类型吗	46
3.4 SpaceMiT IME 矩阵扩展	46
3.4.1 IME 简介	46
3.4.2 vmadot 指令详解	46
3.4.3 分块矩阵乘法	47
3.5 在 Lab2 中应用 RISC-V	47
3.5.1 从 x86 到 RISC-V 的迁移	47
3.5.2 IME 与 AMX 的关键差异	47
3.5.3 适配策略	48
3.6 编译与运行	48

1 Part I: CPU 工程实践

1.1 构建与 Baseline 检视

1.2 引言：从原理到实践

上一篇教程我们建立了 MoE 前向计算的数学模型，推导了算术强度只有约 1 MAC/byte，是典型的访存瓶颈。但“纸上得来终觉浅”，这一章我们要把代码真正跑起来，用真实的计时数据建立性能基线，然后深入到汇编层面，看编译器在 -O3 下到底做了哪些自动向量化，又有哪些它做不到。

本章的所有命令和输出都在 WSL2 Ubuntu 24.04 环境中实测运行。本地开发机是 Intel Core Ultra 7 155H (Meteor Lake)，支持 AVX2 和 AVX-VNNI，但不支持 AVX-512 和 AMX。评测集群使用 Intel Sapphire Rapids，支持 AVX-512 和 AMX。我们在本地跑通流程、建立直觉，在集群上做最终评测。

虽然本地 CPU 不支持 AMX，但这不影响学习流程。-march=sapphirerapids 的编译选项会让编译器为 Sapphire Rapids 生成代码，但 framework (baseline) 始终用通用的 -O3 -g 编译，所以 baseline 在本地和集群上的行为是一致的。

1.3 获取代码与构建

1.3.1 克隆仓库

实验代码位于 ZJUSCT/HPC101 仓库的 src/lab2/ 目录下。在 WSL 中克隆：

```
cd ~
git clone https://github.com/ZJUSCT/HPC101.git
cd ~/HPC101/src/lab2
```

目录结构如下：

```
src/lab2
├── CMakeLists.txt           # 构建配置
├── main.cpp                 # driver (评测时替换为原版)
├── include
│   └── moe.h                # 问题规模、权重结构体、接口声明
├── src
│   ├── moe_ref.cpp          # 标量参考实现 (正确性基准)
│   └── data.cpp             # 数据初始化与正确性检查
└── student
    └── moe_opt.cpp           # ★ 你的代码 (只有这个目录会被收取)
```

记住一条铁律：你只能修改 student/moe_opt.cpp。评测时 student/ 之外的所有文件都会被替换为原版。就算你在别处改了什么，评测时也不会生效。

1.3.2 CMake 构建配置

CMakeLists.txt 中有两个关键设计值得注意。第一个是 framework (baseline) 和 student 的编译选项分离：

```
# framework: 固定用 -O3 -g, 不带任何架构特定标志
add_library(framework OBJECT main.cpp src/moe_ref.cpp src/data.cpp)
target_compile_options(framework PRIVATE -O3 -g)

# student: 允许使用架构特定标志
add_library(student OBJECT student/moe_opt.cpp)
target_compile_options(student PRIVATE -O3 -g)
if(CMAKE_SYSTEM_PROCESSOR MATCHES "x86_64|AMD64|amd64")
    check_cxx_compiler_flag("-march=sapphirerapids" COMPILER_SUPPORTS_SPR)
    if(COMPILER_SUPPORTS_SPR)
        target_compile_options(student PRIVATE -march=sapphirerapids)
    endif()
endif()
```

这个设计确保了 baseline 在所有平台上都是公平的通用 -O3 优化，而你的优化代码可以利用 Sapphire Rapids 的 AVX-512 和 AMX 指令。

第二个设计是 D 和 H 必须是 64 的倍数（在 main.cpp 中校验），这为 AMX tile 对齐和 SIMD lane 对齐铺好了路。

1.3.3 构建与运行

```
cd ~/HPC101/src/lab2
cmake -B build
cmake --build build -j
```

构建成功后，可执行文件在 build/lab2。运行时需要 5 个必选参数和 1 个可选参数：

```
./build/lab2 <num_tokens> <d_model> <d_ff> <num_experts> <top_k> [n_iter]
```

场景	N	D	H	E	K	说明
S1	1	256	128	16	4	单 token 小模型
S2	1	1024	512	16	4	单 token 大模型
S3	128	256	128	16	4	批量小模型
S4	1024	512	128	512	2	大批量大专家数

Table 1: 四个评测场景

1.4 运行 Baseline: 建立性能基线

先看看 driver 的计时逻辑。main.cpp 会对 baseline 和优化版各做 n_iter 次迭代（默认 1000），取总时间计算加速比。计时前有 10 次 warmup（冷缓存、CPU 频率未爬升时跑的）。验证用一个全新 seed 的 token 批次，防止缓存作弊。

直觉 driver 用了 16 个不同的输入批次轮换 (pool = 16)，每次迭代用不同的 x ，避免 CPU 缓存把上一轮的结果“记住”而让计时偏快。这也是为什么 check_result 用一个全新 seed 验证：如果你的优化代码缓存了结果，对新 seed 会直接失败。

在本地 Meteor Lake 上运行四个场景：

```
==== S1 (1 token, D=256, H=128, E=16, K=4, 1000 iter) ===
Baseline time: 0.112768 s
Optimized time: 0.106981 s
Speedup: 1.0541

==== S2 (1 token, D=1024, H=512, E=16, K=4, 1000 iter) ===
Baseline time: 1.89395 s
Optimized time: 1.57191 s
Speedup: 1.20488

==== S3 (128 tokens, D=256, H=128, E=16, K=4, 1000 iter) ===
Baseline time: 10.3108 s
Optimized time: 10.8826 s
Speedup: 0.947456

==== S4 (1024 tokens, D=512, H=128, E=512, K=2, 10 iter) ===
Baseline time: 3.79412 s
Optimized time: 3.61128 s
Speedup: 1.05063
```

注意此时 student/moe_opt.cpp 只是转发调用 moe_forward_ref，所以“优化版”和 baseline 做的事一模一样。加速比不等于 1.0 的原因是：student 用了 -march=sapphirerapids 编译（虽然运行在 Meteor Lake 上），而 framework 只用了 -O3。编译器在 -march=sapphirerapids 下可能生成了一些不同的指令序列，但差异很小。

S4 用了 10 次迭代（而非默认 1000），因为 512 个专家加上 1024 个 token 的规模太大，1000 次迭代会运行很久。实际评测时按需调整。

S3 中“优化版”比 baseline 还慢 (0.95x)，这提醒我们：-march=sapphirerapids 生成的指令在 Meteor Lake 上可能触发微码降频或生成 Meteor Lake 不支持的指令回退路径。在真实集群 (Sapphire Rapids) 上不会有这个问题。这个“负面”结果本身就是一个有教育意义的观察。

1.5 读代码：理解 Baseline 的访存模式

1.5.1 逐 token 遍历的结构

moe_forward_ref 的外层循环是逐 token 遍历：

```
for (int t = 0; t < num_tokens; t++) {
    const float* xt = x + (size_t)t * d_model;
```

```

float* yt = y + (size_t)t * d_model;
// 1. 路由打分 → s[E]
// 2. Top-K 选择 → topk_idx[K]
// 3. 归一化 → gate
// 4. 量化激活 → xq[D]
// 5. 共享专家 + K 个路由专家
// 6. 加权合并 + 残差
}

```

关键问题在第 5 步：对每个 token，它调用 `expert_ffn` 处理共享专家和 K 个被选中的路由专家。每次调用 `expert_ffn` 都要完整读一遍该专家的全部权重（ $3DH$ 个 INT8）。

1.5.2 expert_ffn 的访存模式

回顾 T1 中分析的 S3 场景（ $N = 128, D = 256, H = 128, E = 16, K = 4$ ）：每个 token 选 4 个路由专家 + 1 个共享专家 = 5 个专家。16 个路由专家平均每个被 $128 \times 4 / 16 = 32$ 个 token 选中。也就是说，同一份专家权重被从内存读入 32 次。

直觉 想象一个图书馆（内存），读者（token）轮流来借书（专家权重）。参考实现的方式是：每个读者来了，找到自己要的那 5 本书，逐页读完，放回去。下一个读者来了，可能要借同样的书，又要重新去架上取。按专家分组的方式是：先把所有读者要借的书目汇总，对每本书，一次取下来让所有需要它的读者轮流看。书的取放次数从 128 次降到 16 次。

这就是 T1 中算术强度分析的代码依据：从约 1 MAC/byte（逐 token）提升到约 38 MAC/byte（按专家分组）。

1.6 自动向量化：编译器做了什么

-O3 下编译器会尝试自动向量化循环。我们可以用 `-fopt-info-vec-optimized` 和 `-fopt-info-vec-missed` 两个诊断标志来看它做了什么。手动编译 `moe_ref.cpp`：

```

g++ -O3 -g \
    -fopt-info-vec-optimized \
    -fopt-info-vec-missed \
    -I include -c src/moe_ref.cpp -o /tmp/moe_ref.o

```

输出的诊断信息（节选关键行）：

```

src/moe_ref.cpp:53:23: missed: not vectorized: control flow in loop
src/moe_ref.cpp:55:27: optimized: loop vectorized using 16 byte vectors
src/moe_ref.cpp:39:39: missed: statement clobbers memory: expf(...)
src/moe_ref.cpp:49:31: missed: statement clobbers memory: lrintf(...)
src/moe_ref.cpp:69:23: missed: loop nest containing two or more
                        consecutive inner loops cannot be vectorized
src/moe_ref.cpp:120:27: optimized: loop vectorized using 16 byte vectors

```


1.6.1 诊断解读

逐行来看这些诊断信息的含义：

行号	位置	结果	原因
53	gate/up 内积外层	missed	循环内有控制流 (if (a > h_amax))
55	内积内部累加	optimized	向量化为 16 字节 (SSE 128 位)
39	SiLU 中 expf	missed	expf 是外部调用, clobbers memory
49	量化 lrintf	missed	lrintf 是外部调用, clobbers memory
69	down 投影内积	missed	嵌套循环无法向量化
120	路由权重归一化	optimized	向量化为 16 字节

Table 2: 自动向量化诊断解读

直觉 三个关键发现：第一，最耗时的内积循环（gate/up 和 down）要么没有被向量化（因为控制流或嵌套结构），要么只向量化为 16 字节 SSE 而非 256 位 AVX2。第二，expf 和 lrintf 这种 C 库调用会阻止向量化，因为编译器不知道它们是否有副作用。第三，“16 byte vectors”意味着编译器在用 SSE2（128 位），而非 AVX2（256 位），因为 framework 没有指定 -march。

1.6.2 为什么“16 byte vectors”

framework 的编译选项是 -O3 -g，没有 -march 标志。这意味着编译器只使用 x86-64 的基准指令集（SSE2），不会自动使用 AVX2 或 AVX-512。SSE2 的向量宽度是 128 位 = 16 字节，所以诊断信息中反复出现“16 byte vectors”。

如果给 framework 加上 -mavx2，诊断信息会变成“32 byte vectors”，性能也会有提升。但实验刻意不给 baseline 加架构标志，确保它是一个公平的、可移植的基准。你的优化代码在 student 中用 -march=sapphirerapids，可以使用 AVX-512（512 位 = 64 字节）和 AMX。

1.7 汇编检视：objdump

-fopt-info-vec 告诉我们哪些循环被向量化了，但没告诉我们编译器生成了什么指令。objdump -d 可以反汇编目标文件，看到实际的机器指令。

```
objdump -d -C --no-show-raw-insn \  
build/CMakeFiles/framework.dir/src/moe_ref.cpp.o \  
| less
```

-C 把 C++ 名称修饰（name mangling）还原为可读的函数签名，--no-show-raw-insn 隐藏原始字节只显示助记符。

找到 expert_ffn 函数，定位到内积循环的向量化部分。以下是实际汇编的节选（注释标出关键指令）：

```
; 加载 16 字节 int8 数据 (128 位 SSE)  
e8: movdqu (%rcx,%rax,1),%xmm0 ; w_gate[f*D + d..d+15] -> xmm0  
ed: movdqu (%rbx,%rax,1),%xmm1 ; xq[d..d+15] -> xmm1
```

```

; int8 符号扩展到 int16 (SSE 没有 int8×int8→int32 一步指令)
f2: movdqa %xmm7,%xmm10
f7: movdqa %xmm7,%xmm3
fb: pcmpgtb %xmm0,%xmm10      ; 生成 w_gate 的符号掩码
100: pcmpgtb %xmm1,%xmm3      ; 生成 xq 的符号掩码
104: movdqa %xmm0,%xmm9
109: movdqa %xmm1,%xmm8
10e: punpcklbw %xmm3,%xmm8    ; 低 8 字节 int8 -> int16
113: punpcklbw %xmm10,%xmm9
118: punpckhbw %xmm10,%xmm0   ; 高 8 字节 int8 -> int16
11d: pmullw %xmm8,%xmm9      ; int16 乘法 (只乘了一半)
...
139: punpcklwd %xmm10,%xmm3   ; int16 -> int32
13e: paddb %xmm4,%xmm3       ; int32 累加

```

1.7.1 汇编解读

这段汇编展示了编译器自动向量化的局限性，有三个关键问题：

1. **只用 SSE 128 位。**movdqu / pmullw / paddb 都是 XMM (128 位) 指令，每次处理 16 个 INT8。如果用 AVX2 的 YMM (256 位)，一次能处理 32 个；用 AVX-512 的 ZMM (512 位)，一次 64 个。
2. **没有 VNNI 指令。**INT8 × INT8 → INT32 的点积需要三步：pcmpgtb (符号判断) → punpcklbw (int8→int16) → pmullw (int16 乘法) → punpcklwd (int16→int32) → paddb (累加)。而 VNNI 的 vdpbusd 一条指令就完成了 int8×int8→int32 的乘加。
3. **sign extension 开销大。**因为 INT8 可能是负数 (-128 到 127)，编译器必须先判断符号再扩展，这消耗了大量指令周期。

直觉 把这段汇编和 VNNI 指令对比：编译器自动生成的代码用约 10 条指令处理 16 个 INT8 元素的乘加，而一条 vdpbusd 就能处理 32 个 INT8 元素的乘加并累加到 INT32。效率差距可达 20 倍以上。这就是为什么要手写 SIMD 乃至使用 AMX。

1.8 Compiler Explorer: 在线对比

除了 objdump, [Compiler Explorer](https://godbolt.org/) (简称 godbolt) 是一个更友好的汇编检视工具。它把源码和汇编并排显示，鼠标悬停某行代码就能高亮对应的汇编。

1.8.1 基本使用流程

1. 打开 <https://godbolt.org/>
2. 左侧粘贴 moe_ref.cpp 的内容 (连同 #include 部分)
3. 右侧编译器选 x86-64 gcc 13 (与本地版本一致)
4. 编译选项填 -O3 -g, 观察汇编
5. 对比加上 -mavx2 或 -march=sapphirerapids 后汇编的变化

Compiler Explorer 不需要登录，也不需要包含 `moe.h`（把结构体定义粘贴进去即可）。但注意 `moe.h` 中有 `MAX_NUM_TOKENS` 等宏定义，如果不粘贴会导致编译失败。最简单的方式是把 `moe.h` 的内容粘贴在 `.cpp` 文件顶部。

1.8.2 对比不同编译选项

在 Compiler Explorer 中可以同时开多个编译器窗口对比。推荐对比以下组合：

选项	向量宽度	关键指令
-O3 (baseline)	128 位 SSE	<code>pmullw + paddb</code>
-O3 -mavx2	256 位 AVX2	<code>vpmullw + vpaddb</code>
-O3 -march=sapphirerapids	512 位 AVX-512	<code>vpmullw ZMM + AVX-VNNI</code>
-O3 -mavx2 -mavxvnni	256 位 + VNNI	<code>vpdpbusd</code> （一步乘加）

Table 3: 不同编译选项的向量化效果

直觉 你会看到从 -O3 到 -mavx2 -mavxvnni，`pmullw`（int16 乘法）序列消失，取而代之的是一条 `vpdpbusd`。这就是 VNNI 的威力：把 `sign extension + int16 乘法 + int32 累加` 压成一条指令。后续教程 T3 会手写 VNNI intrinsics。

1.9 环境差异说明

本教程的命令和输出在本地 Meteor Lake 上运行。与评测集群（Sapphire Rapids）的关键差异：

项目	本地 (Meteor Lake)	集群 (Sapphire Rapids)	影响
SIMD	AVX2 + AVX-VNNI	AVX-512 + VNNI + AMX	集群可做更深向量化
核心/线程	11C/22T	视分配而定	多线程行为不同
L3 缓存	24 MiB	更大	S4 大专家场景缓存行为不同
baseline 行为	一致（-O3 通用）	一致	公平基准

Table 4: 本地与集群环境对比

baseline 在两个平台上的行为是一致的（都用通用 -O3），所以本地测的加速比趋势与集群上大致吻合。但绝对性能数字会不同，因为集群的 AVX-512 和 AMX 能让你的优化代码跑得更快。

1.10 数据布局与 Preprocess

1.11 引言：为什么数据布局决定了优化上限

T3 中我们学会了用 VNNI 指令做 INT8 点积，一条 `vpdpbusd` 顶十条标量指令。但如果权重数据在内存中的排列方式不利于向量加载，VNNI 的效率会被低效访存拖垮。这就好比你有了一辆跑车，但路面的坑洼让它只能挂一档。

本章讨论 Lab2 中 `preprocess` 函数的用途：在计时开始前，把权重重排成对 SIMD 和缓存友好的布局。这个函数不计入计时，是你“磨刀”的地方。

1.12 Baseline 的访存模式

1.12.1 逐 token 遍历的结构

回顾 T1 的分析：moe_forward_ref 的外层循环逐 token 遍历，对每个 token 调用 expert_ffn 处理共享专家和 K 个路由专家。关键问题是：**每个 token 都要完整读一遍所选专家的全部权重。**

以 S3 场景 ($N = 128, D = 256, H = 128, E = 16, K = 4$) 为例，16 个路由专家平均每个被 $128 \times 4 / 16 = 32$ 个 token 选中。同一份专家权重被从内存读入 32 次。

直觉 这就像一个图书馆员，每次有读者来都重新去书架上取同一本书。如果先把所有读者要借的书目汇总，对每本书一次取下来让所有需要的读者轮流看，书的取放次数从 128×5 次降到 17 次（16 路由专家 + 1 共享专家）。

1.12.2 expert_ffn 的按行访问

在 expert_ffn 中，gate 投影的访存模式是：

```
for (int f = 0; f < d_ff; f++) {
    for (int d = 0; d < d_model; d++) {
        acc_g += w_gate[f * d_model + d] * xq[d];
    }
}
```

权重 w_gate 的布局是行主序 $[d_ff][d_model]$ ，第 f 行的起始地址是 $f * d_model$ 。对每个输出元素 f ，内层循环连续读 D 个 INT8。这种布局对标量代码没问题，但对 SIMD 加载有几个隐患：

1. **跨行跳跃**：从 f 行到 $f + 1$ 行的地址跳了 D 字节。如果 D 不是缓存行大小的整数倍，会产生跨行访存
2. **VNNI 友好性**：T3 中 VNNI 一次处理 32 字节（AVX2）或 64 字节（AVX-512），如果 D 是 64 的倍数，加载是对齐的；但如果没有分组，每个 token 都要重新加载权重，缓存复用率低
3. **AMX tile 要求**：AMX 的 tile 是 2D 矩阵，要求特定的布局才能高效加载，这在 T5 中展开

1.13 preprocess：你的磨刀石

preprocess 函数在计时开始前被调用一次，你可以在这里对权重做任何重排、预计算或预分配：

```
void preprocess(MoEWeights& w) {
    // 这里可以对 w 做重排/预打包
    // 这段代码不计入计时
}
```

`MoEWeights` 的形状字段 (`d_model`、`d_ff`、`num_experts`、`top_k`) 由 *driver* 在调用 *preprocess* 之前填入。你的 *preprocess* 和 *moe_forward_optimized* 从中读取。因此你的实现不能假设任何维度取固定值。

preprocess 中可以做的事情包括：

1. **按专家分组**：重排计算顺序，从“逐 token”变成“逐专家”
2. **权重转置/分块**：把权重布局成对 SIMD 加载友好的形式
3. **预计算行和**：T3 中偏移技巧所需的 `w_rowsum`
4. **对齐分配**：用 `aligned_alloc` 分配对齐到 64 字节的权重缓冲区
5. **偏移权重**：如果用 `vdpbusd` 方案，可以预计算权重的无符号版本

1.14 按专家分组：从逐 token 到逐专家

这是 Lab2 优化中最重要的一步。核心思想是改变循环结构：外层循环遍历专家，内层循环遍历所有选中该专家的 token。

1.14.1 原始结构（逐 token）

```
for token t = 0..N:
    quantize x[t]
    expert_ffn(shared, x[t])           // 读 shared 权重
    for k = 0..K:
        e = topk[t][k]
        expert_ffn(e, x[t])           // 读 expert e 的权重
    combine
```

权重读取次数：每个 token 读 $(1 + K)$ 个专家，总读取次数 $N \times (1 + K)$ 。每个专家平均被读 $N \times K / E$ 次。

1.14.2 分组结构（逐专家）

```
for expert e = 0..E:
    // 收集选中 e 的 token 列表
    tokens_e = [t for t in 0..N if e in topk[t]]
    for t in tokens_e:
        expert_ffn(e, x[t])           // 权重已在缓存中
// shared expert 对所有 token 执行
for token t = 0..N:
    expert_ffn(shared, x[t])
```

权重读取次数：每个专家权重只从内存读一遍 ($E + 1$ 次)，后续命中缓存。

例 以 S3 场景 ($N = 128, D = 256, H = 128, E = 16, K = 4$) 为例：

逐 token：每个 token 读 5 个专家 $\times 3DH = 491\,520$ 字节。总访存 $\approx 128 \times 491\,520 \approx 63\text{ MiB}$ 。

逐专家：每个专家权重 $3DH = 98\,304$ 字节，读一遍。总访存 $\approx 17 \times 98\,304 \approx 1.67$ MiB。

访存从 63 MiB 降到 1.67 MiB，减少约 38 倍。算术强度从约 1 MAC/byte 提升到约 38 MAC/byte。

1.14.3 实现要点

分组需要先确定每个 token 选中了哪些专家。路由打分和 Top-K 选择必须在 `moe_forward_optimized` 中进行（因为 `x` 是运行时输入），但可以在第一次遍历时收集分组信息：

```
// 第一遍：路由打分 + Top-K + 量化激活
for (int t = 0; t < num_tokens; t++) {
    // 计算 affinity, topk, gate, xq
    // 记录 token t 选中了哪些专家
    for (int k = 0; k < top_k; k++) {
        int e = topk_idx[t][k];
        expert_tokens[e].push_back(t);
    }
    // 存储 xq[t] 和 gate[t] 供第二遍使用
}

// 第二遍：按专家分组执行
for (int e = 0; e < num_experts; e++) {
    for (int t : expert_tokens[e]) {
        expert_ffn(e, xq[t], ...); // 权重 e 在缓存中
    }
}

// 共享专家
for (int t = 0; t < num_tokens; t++) {
    expert_ffn(shared, xq[t], ...);
}

// 第三遍：加权合并 + 残差
for (int t = 0; t < num_tokens; t++) {
    y[t] = x[t] + o_shared[t] + sum(gate * o_routed[t]);
}
```

三遍遍历增加了代码复杂度，但第一遍和第三遍的计算量（路由打分、量化、合并）相对很小，瓶颈在第二遍的专家 FFN。分组让第二遍的权重读取量降低 38 倍，是绝对的正优化。

1.15 权重布局优化

分组解决了“权重被反复读取”的问题，但权重的内存布局仍然影响 SIMD 加载效率。以下是几种常见的布局优化。

1.15.1 对齐分配

SIMD 加载指令（如 `_mm256_loadu_si256`）虽然支持未对齐访问，但对齐访问更高效。可以在 preprocess 中用 `aligned_alloc` 分配对齐到 64 字节的权重缓冲区：

```
void* buf = aligned_alloc(64, num_experts * d_ff * d_model);
```

然后将原始权重复制过来。64 字节对齐同时满足 AVX-512（64 字节）和 AMX（通常 64 字节）的要求。

1.15.2 权重预偏移

如果用 T3 的方案 B（vpdpbusd 偏移技巧），可以在 preprocess 中预计算：

1. 权重的行和 `w_rowsum[e][f]`：第 e 个专家第 f 行的权重和，用于补偿
2. 激活的偏移版本：不需要在 preprocess 中做（因为 x 是运行时输入），但在量化后立即偏移

```
// preprocess 中预计算每行权重的和
for (int e = 0; e < num_experts; e++) {
    for (int f = 0; f < d_ff; f++) {
        int32_t sum = 0;
        for (int d = 0; d < d_model; d++) {
            sum += w_gate[(size_t)e * d_ff * d_model + f * d_model + d];
        }
        w_gate_rowsum[e][f] = sum; // 用于运行时补偿
    }
}
```

1.15.3 转置 down 投影权重

`expert_ffn` 中的 down 投影访问 `w_down[d * d_ff + f]`，即按行遍历输出维度 d ，内层循环遍历 f 。但权重布局是 `[d_model][d_ff]`，对每个 d 连续读 H 个元素。

如果要想 VNNI 同时处理多个输出 d （寄存器分块），就需要同时加载 `w_down` 的多行。把 `w_down` 转置成 `[d_ff][d_model]` 布局，可以让多个 d 共享同一次激活加载，提高数据复用。

直觉 寄存器分块的核心思想是：把同一次加载的输入数据“广播”给多个输出。如果激活 h_q 只加载一次，就能同时计算多个 d 的输出，就不需要为每个 d 重新加载 h_q 。这要求权重按 d 维度分块排列，让多个 d 的权重连续存储。

1.16 缓存层级感知

分组后，每个专家的权重为 $3DH$ 字节。以 S3 场景为例， $3 \times 256 \times 128 = 98\,304$ 字节 ≈ 96 KiB。

缓存层级	容量 (Meteor Lake)	能否放下单个专家
L1d	48 KiB / 核	放不下
L2	2 MiB / 核	放得下
L3	24 MiB (共享)	放得下所有专家

Table 5: 缓存层级与专家权重大小

单个专家的权重 (96 KiB) 放不进 L1d (48 KiB)，但放得进 L2 (2 MiB)。这意味着分组后，每个专家的权重在第一次访问时从 L3 加载到 L2，后续的 token 可以从 L2 命中，延迟远低于 DRAM。

S4 场景 ($D = 512, H = 128, E = 512$) 的挑战在于专家数量多 (512 个)，总权重量大。但每个 token 只选 2 个专家，分组后仍有很高的复用率。关键是确保分组后的 token 列表不会让缓存频繁在不同专家间跳跃。

1.17 分组 + VNNI 的完整框架

把分组（本章）和 VNNI (T3) 结合起来，moe_forward_optimized 的框架大致如下：

```
void moe_forward_optimized(const float* x, const MoEWeights& w,
                          float* y, int num_tokens) {
    // 第一遍：路由 + 量化 + 分组
    for (int t = 0; t < num_tokens; t++) {
        compute_affinity(x[t], w, s);
        topk_select(s, w.bias, topk_idx[t]);
        normalize_gate(s, topk_idx[t], gate[t]);
        quantize(x[t], xq[t], s_x[t]);
        offset_to_uint8(xq[t], xq_u[t]); // T3 偏移技巧
        for (int k = 0; k < w.top_k; k++)
            expert_tokens[topk_idx[t][k]].push_back(t);
    }

    // 第二遍：按专家分组执行 (VNNI 加速)
    for (int e = 0; e < w.num_experts; e++) {
        for (int t : expert_tokens[e]) {
            vnni_expert_ffn(w, e, xq_u[t], s_x[t], o[t]);
            // 权重 e 在 L2 缓存中
        }
    }

    // 共享专家
    for (int t = 0; t < num_tokens; t++)
        vnni_expert_ffn(w, SHARED, xq_u[t], s_x[t], o_shared[t]);

    // 第三遍：加权合并
    for (int t = 0; t < num_tokens; t++) {
        y[t] = x[t] + o_shared[t];
        for (int k = 0; k < w.top_k; k++)
            y[t] += gate[t][k] * o[topk_idx[t][k]][t];
    }
}
```



```

    }
}

```

`vnni_expert_ffn` 是 T3 中用 VNNI 替换了内积循环的 `expert_ffn`，用 `vpdpbusd` 或 `vpdpwssd` 做点积，配合 `preprocess` 中预计算的 `w_rowsum` 做偏移补偿。

1.18 VTune 采样与 GUI 分析

1.19 引言：为什么要用 Profiler

前面几章我们写了分组、VNNI、AMX 的代码，但怎么知道这些优化是否有效？瓶颈在哪里？下一步该优化什么？靠猜测和穷举各种优化方式是痛苦的。**Profiler**（性能分析器）能给你一个可解释的指标，让优化过程有方向感。

Lab2 文档推荐使用 Intel VTune Profiler。本章介绍 VTune 的基本使用流程：在集群上命令行采样，下载结果到本地，用 GUI 分析。

直觉 Profiler 的价值不在于告诉你“哪里慢”，而在于告诉你“为什么慢”。一个优化改动不管是正优化还是负优化，有一个可解释的指标会给你较强的正反馈，避免陷入“炼丹”的痛苦。

1.20 VTune 分析类型概览

VTune 提供多种分析类型，每种关注不同的性能维度。Lab2 中最常用的三种：

分析类型	回答什么问题	Lab2 中的用途
Hotspots	时间花在了哪些函数/代码行	定位热点循环
Microarchitecture Exploration	流水线为何停顿	解释 Back-End Bound
Memory Access	缓存命中率、带宽利用	确认访存瓶颈

Table 6: VTune 分析类型与用途

本章聚焦 Hotspots 分析和基本 GUI 操作。Microarchitecture Exploration 和 Roofline 分析在 T7 中展开。

1.21 集群上的命令行采样

1.21.1 环境准备

按照集群说明进入计算节点并加载 VTune 环境。VTune 目前可能无法在容器内运行，需要在计算节点上直接使用。

VTune 采样需要内核访问权限（读取硬件性能计数器），容器内通常没有这个权限。如果集群使用 *Slurm*，通过 *srun* 或 *sbatch* 在计算节点上运行 VTune 命令。

1.21.2 Hotspots 采样

```
vtune -collect hotspots \  
      -result-dir vtune-hotspots \  
      -- ./build/lab2 128 256 128 16 4 2000
```

这条命令的含义：

1. -collect hotspots: 采样类型为热点分析
2. -result-dir vtune-hotspots: 结果保存到 vtune-hotspots 目录
3. --: 后面的命令是被分析的目标程序
4. 128 256 128 16 4 2000: S3 场景的参数，最后的 2000 是迭代次数

直觉 为什么要设 2000 次迭代？因为 VTune 的 Hotspots 分析是通过采样工作的：它在固定时间间隔抓取 CPU 正在执行的指令地址。如果程序运行时间太短，采样点太少，统计结果不可靠。默认 1000 次迭代可能不够，2000 次更稳妥。

1.21.3 Microarchitecture Exploration 采样

```
vtune -collect uarch-exploration \  
      -result-dir vtune-uarch \  
      -- ./build/lab2 128 256 128 16 4 2000
```

这条命令做微架构探索分析，读取硬件性能计数器，给出 Top-down 分类的结果。T7 会详细解读。

1.21.4 采样注意事项

1. **迭代次数要足够**：让被测部分运行时间至少几秒，否则采样点太少
2. **输入一致**：每次采样用相同的输入参数，确保结果可比较
3. **绑核一致**：如果绑核，每次采样的绑核方式要相同
4. **同时保存可执行文件**：本地 GUI 需要和采样时一致的可执行文件来显示源码和汇编

1.22 下载结果到本地

采样结束后，需要下载**完整的结果目录**（不要只复制 .vtune 文件）和采样时使用的可执行文件：

```
# 在本地终端执行  
scp -r <username>@<cluster>:<path-to-repo>/lab2/vtune-hotspots .  
scp <username>@<cluster>:<path-to-repo>/lab2/build/lab2 .
```

为什么需要可执行文件？VTune GUI 显示源码和汇编视图时，需要从可执行文件中读取调试信息（-g 编译时保留的 DWARF 信息）。如果本地没有与采样时一致的可执行文件，源码视图会显示空白。

1.23 本地 GUI 分析

1.23.1 打开结果

1. 启动本地 VTune Profiler GUI
2. 选择 **File > Open > Result...**
3. 浏览到下载的 vtune-hotspots 目录
4. 打开其中的 .vtune 文件

1.23.2 Binary/Symbol Search

如果源码或汇编视图无法正确显示，需要在 VTune 中补充可执行文件的搜索路径：

1. **Tools > Options > Binary/Symbol Search**
2. 添加下载的可执行文件 lab2 所在的目录
3. 重新打开结果

1.23.3 Bottom-up 视图

Hotspots 分析默认显示 **Bottom-up** 视图，从最耗时的函数向上追溯调用者：

Function / Call Stack	CPU Time	CPU Utilization
→ expert_ffn	85%	1.2 core
→→ moe_forward_ref	85%	
→→→ main		
→ init_data	8%	
→ check_result	3%	

Listing 1: Bottom-up 视图示意（典型结果）

可以看到 expert_ffn 占了 85% 的时间，这验证了 T1 的分析：内积循环是热点。init_data 和 check_result 占比很小，不需要优化。

1.23.4 源码视图

在 Bottom-up 视图中双击 expert_ffn，会跳转到源码视图。VTune 把采样点映射到源码行，每行显示 CPU Time 占比：

Source Line	CPU Time	Code
acc_g += w_gate[...] * xq[d];	42%	内积累加
acc_u += w_up[...] * xq[d];	40%	另一内积
float vg = (float)acc_g * ...;	2%	反量化
float silu = vg / (1.0f + expf(-vg));	1%	SiLU

Listing 2: 源码视图示意（典型结果）

直觉 源码视图告诉你“哪一行最耗时”。在 baseline 中，内积累加行（acc_g += ...）占了大头，这验证了编译器没能有效向量化这些循环（T2 的分析）。优化后，你应该在源码

视图中看到热点从内积循环“消失”（被 VNNI/AMX 替代），转移到其他部分（如 SiLU、合并等）。

1.23.5 汇编视图

在源码视图中可以切换到汇编视图，看到每条指令的采样次数。这对验证 VNNI/AMX 是否生效很有用：优化后，你应该看到 `vpdpbusd` 或 `tdpbssd` 指令出现在热点行，而不是 T2 中的 `pmullw + paddb` 序列。

1.24 Hotspots 分析实战

1.24.1 Baseline 的 Hotspots

对 baseline 做一次 Hotspots 采样，预期看到：

1. `expert_ffn` 占 80-90% 的时间
2. 热点行集中在 `gate/up` 和 `down` 投影的内积循环
3. `expf` 和 `lrintf` 调用占少量但不可忽略的时间

这验证了 T1-T2 的分析：内积循环是瓶颈，需要向量化优化。

1.24.2 优化版本的 Hotspots

对优化版本做 Hotspots 采样，观察热点是否转移：

1. 如果 VNNI/AMX 生效，内积循环的热点应该大幅降低
2. 新的热点可能出现在 SiLU/反量化/合并等 FP32 环节
3. 如果热点仍在内积循环，说明 VNNI/AMX 没有生效（检查编译标志）

一个常见的失败模式：代码中写了 *VNNI intrinsic*，但编译标志没有包含 `-mavxvnni` 或 `-march=sapphirerapids`，导致编译器回退到标量代码。Hotspots 分析能快速发现问题：如果内积循环仍然是热点，说明 VNNI 没有生效。

1.24.3 对比分析

VTune 支持同时打开多个结果做对比。把 baseline 和优化版本的 Hotspots 结果并排打开，比较：

1. **总时间**：优化版是否更快
2. **热点分布**：热点是否转移到了其他代码
3. **指令类型**：汇编视图中是否出现了 VNNI/AMX 指令

1.25 优化迭代方法论

1.26 引言：把工具箱串成迭代闭环

前面七章分别讲解了原理（T1）、构建与汇编（T2）、VNNI（T3）、分组与布局（T4）、AMX（T5）、VTune 采样（T6）和 Top-down 分析（T7）。本章把这些工具串成一个完整的优化迭代闭环，并提供一个推荐的优化路径和迭代日志模板，帮助你系统地完成 Lab2。

直觉 优化不是“把所有技巧都试一遍”，而是“假设 → 实现 → 验证 → 分析 → 下一步”的循环。每一步都有一个可解释的指标（加速比、Top-down、Roofline），让你知道“做了什么”“效果如何”“为什么”“下一步做什么”。

1.27 优化迭代方法论

1.27.1 四步循环

每次优化遵循四步：

```
假设：提出一个优化假设（如“分组能降低访存”）
      →
实现：写代码（分组循环 + VNNI）
      →
验证：跑 check_result 确认正确性，跑 driver 得加速比
      →
分析：用 VTune 采样，看 Top-down 变化，判断是否验证了假设
      → 回到假设
```

Listing 3: 优化迭代四步循环

1.27.2 每次改动记录什么

建议维护一份迭代日志，每次改动记录：

1. **改动描述**：做了什么（如“按专家分组 + VNNI 方案 A”）
2. **正确性**：check_result 是否通过，RMSE 值
3. **加速比**：四个场景的 Speedup
4. **Profiler**：Hotspots 热点是否转移，Top-down 哪个指标变化
5. **解释**：为什么有效（或为什么无效），是否验证了假设
6. **下一步**：根据分析选择下一个优化方向

1.28 推荐优化路径

1.28.1 Step 0: Baseline（已完成）

T2 中已跑通。四个场景的加速比约为 1.0x（因为 moe_opt.cpp 只是转发调用 moe_forward_ref）。建立了性能基线和 Profiler 画像。

1.28.2 Step 1: 按专家分组

T4 中讲解的三遍遍历框架。不改变内积的计算方式，只改变循环结构。

场景	预期加速比	正确性	Top-down 变化
S1	$\approx 1x$	PASS	无变化（单 token 无分组收益）
S2	$\approx 1x$	PASS	无变化（单 token）
S3	$\approx 3 - 5x$	PASS	DRAM Bound $\rightarrow$ L2 Bound
S4	$\approx 2 - 4x$	PASS	DRAM Bound $\rightarrow$ LLC Bound

Table 7: Step 1: 按专家分组预期效果

S1 和 S2 只有 1 个 token，分组没有收益（每个专家只被 1 个 token 选中）。分组主要在 S3（128 token）和 S4（1024 token）中生效。

1.28.3 Step 2: VNNI 方案 A（int16 扩展）

T3 中的方案 A：用 `_mm256_cvtepi8_epi16` 扩展到 int16，再用 `vpdpwssd` 做点积。这是最简单的 VNNI 方案，不涉及偏移技巧。

场景	预期加速比	正确性	关键变化
S1	$\approx 1.5 - 2x$	PASS	Retiring 升高
S2	$\approx 2 - 3x$	PASS	Core Bound 上升
S3	$\approx 5 - 8x$	PASS	分组+VNNI 叠加
S4	$\approx 4 - 6x$	PASS	同上

Table 8: Step 2: VNNI 方案 A 预期效果

1.28.4 Step 3: VNNI 方案 B（偏移技巧）

T3 中的方案 B：激活偏移到 uint8，用 `vpdpbusd` 替代 `vpdpwssd`。吞吐量翻倍（int8 比 int16 多一倍），但需要预计算 `w_rowsum` 和运行时偏移。

预期在 Step 2 基础上再提升 $1.5 - 2x$ 。

1.28.5 Step 4: AMX

T5 中的 AMX tile 矩阵乘。用 `_tile_dpbssd` 替代 VNNI 点积，一条指令做 16 384 个乘加。

预期在 Step 3 基础上再提升 $2 - 4x$ （取决于 tile 利用率和内存带宽）。

1.28.6 Step 5: 多线程（可选）

如果单线程已接近计算峰值，多线程可以进一步利用多核。用 OpenMP 或 `std::thread` 把外层循环（按专家分组）分到不同线程。

多线程的注意事项：确保每个线程处理不同的专家（避免写冲突），注意 NUMA 亲和性（线程访问本地内存更快），用 VTune 的 *Threading* 分析检查负载均衡和同步开销。

1.29 迭代日志模板

```
==== Iteration N: [改动名称] ====
日期: 2025-xx-xx
改动: [一句话描述]
```

假设：[预期效果]

正确性：

S1: PASS (RMSE: 0.000xxx)
S2: PASS (RMSE: 0.000xxx)
S3: PASS (RMSE: 0.000xxx)
S4: PASS (RMSE: 0.000xxx)

加速比：

S1: x.xxx (baseline: x.xxs -> opt: x.xxs)
S2: x.xxx (baseline: x.xxs -> opt: x.xxs)
S3: x.xxx (baseline: x.xxs -> opt: x.xxs)
S4: x.xxx (baseline: x.xxs -> opt: x.xxs)

VTune (S3):

Hotspots: [热点函数名] 占 xx%
Top-down: Retiring xx%, FE Bound xx%, Bad Spec xx%, BE Bound xx%
IPC: x.xx
L2 命中率: xx%

解释：[为什么有效/无效]

下一步：[下一个优化方向]

直觉 迭代日志的价值不在于“记录”，而在于“思考”。当你写下“为什么有效”时，你会发现自己对系统的理解在加深。当你写下“下一步”时，你不是在盲目试错，而是在基于分析做出有依据的决策。

1.30 性能上限分析

当主要热点已经稳定后，用 T7 的 Roofline 方法评估剩余优化空间：

1. 估算运算量 $N \times (K + 1) \times 3DH$ (MAC 数)
2. 估算数据搬运量 (分组后 $\approx (E + 1) \times 3DH$ 字节 + 激活)
3. 计算算术强度，在 Roofline 图中定位
4. 比较 P / P_{peak} (当前性能 / 计算峰值)
5. 如果 $> 80\%$ ，剩余空间有限，考虑多线程
6. 如果 $< 50\%$ ，还有较大空间，用 Top-down 找瓶颈

1.31 常见陷阱与解决方案

陷阱	症状	解决方案
VNNI 未生效	内积仍是热点	检查 <code>-march</code> 或 <code>-mavxvnni</code>
分组后缓存抖动	S4 加速比不如 S3	减小工作集，调整分块大小
AMX tile 布局错误	SIGILL 或结果错误	检查 <code>stride</code> 和对齐
偏移补偿遗漏	RMSE 超阈值	确认 <code>w_rowsum</code> 预计算正确
水平求和错误	正确性检查失败	用标量验证 VNNI 结果
多线程 NUMA	多线程反而变慢	绑核 + NUMA 亲和
AVX-512 降频	512 位反而更慢	实测比较 256/512 位

Table 9: 常见陷阱与解决方案

最隐蔽的陷阱是“偏移补偿遗漏”：用了 `vpdpbusd` 偏移技巧，但忘了在结果中减去 $128 * w_rowsum$ ，导致 RMSE 超阈值。`check_result` 会报错，但错误信息只说“不正确”，不直接指出原因。建议用 T3 中的三维点积例子做单元测试，先验证 VNNI 点积的正确性，再集成到 `expert_ffn` 中。

1.32 填充 lab2.typ 报告

完成优化后，把结果填入 `labs/lab2.typ` 报告。各章节与本系列教程的对应关系：

lab2.typ 章节	对应教程
实验目的	T1 概述
MoE 前向计算原理	T1
代码框架与接口	T2
Baseline 分析	T1 + T2
数据布局重排	T4
自动向量化与手写 SIMD	T2 + T3
Intel AMX 矩阵加速	T5
其他优化	T4 + T8
性能评测	T2 baseline + T8 迭代结果
Profiler 分析	T6 + T7
思考题	T1（题 1、3）+ T3（题 2）
附录：优化代码	T3 + T4 + T5
附录：VTune 命令	T6

Table 10: lab2.typ 章节与教程对应

2 Part II: GPU 工程实践

2.1 TileLang 语法与编程模型

2.2 引言：写 CUDA 太繁琐

Lab3 要实现 GDN 的 prefill forward，最直接的方式是写 CUDA Kernel。但只要你写过哪怕一次完整的 CUDA GEMM，就会感受到那种繁琐：分配 Shared Memory、设计 XOR Swizzle 避免 Bank Conflict、手动启动 cp.async 异步搬运、用 __pipeline_memcpy_async 与 __pipeline_wait_prior 维护多级流水线、然后正确发起 mma.sync 或 wmma 调用 Tensor Core。一个像样的 Level-2 GEMM 至少需要几百行代码，而且每个细节出错都会让性能崩塌。

更糟的是，这些细节里没有一条是算法本身。你想表达的是“取一个 16×16 的 A tile、一个 16×16 的 B tile、做矩阵乘累加到 C tile”，而不是“在 Shared Memory 里用 XOR Swizzle 排布、用 mbarrier 等待 cp.async 完成、用 WGMMA descriptor 描述 SMEM 地址”。**算法与硬件细节**被强行耦合在一起，让高性能 Kernel 的门槛高得离谱。

我们需要一种更高层的抽象：既能描述“分块计算”的算法意图，又能让编译器去处理 Thread Mapping、Swizzle、异步搬运这些底层细节。**TileLang**（瓦片语言）就是为此而生。

直觉 不妨把写 CUDA 想象成“亲手组装一辆 F1 赛车”：你要懂引擎、悬挂、空气动力学，每个螺丝都要自己拧。TileLang 则像是“开一辆调校好的赛车”：你只管踩油门、打方向盘，引擎的内部协调交给车载电脑。当然，要拿到极致性能，你还是得理解引擎在做什么，但至少不会因为一个螺丝没拧紧就退赛。

2.3 TileLang 是什么

TileLang 是一种面向高性能计算的**领域专用语言**（Domain-Specific Language, DSL）。它把 GPU Kernel 的开发抽象到“tile 操作”这一层：你只需要描述“哪个 tile 做什么计算”，编译器会自动处理 Thread Mapping、内存层级、Tensor Core 调用、流水线同步等细节。

TileLang 的定位介于 CUDA 与 Triton/MLIR 之间：

1. 比 CUDA 高：你不用手写 __shared_memory__、__syncthreads()、mma.sync 等底层指令。
2. 比 Triton 更显式：TileLang 保留 tile 的层次结构，让你能精细控制 Shared Memory 与寄存器的分配，对 Tensor Core 的调用也更直接。
3. 与 MLIR 同源：TileLang 在底层会生成 MLIR/TVM 的中间表示，再 lowering 到 CUDA 或 LLVM。

TileLang 的官方文档在 tilelang.com，示例仓库在 github.com/tile-ai/tilelang。Lab3 的环境已预装好 TileLang，可以用 `python -c "import tilelang; print(tilelang.__version__)"` 查看版本。

TileLang 的核心设计理念是 **tile-based programming**（瓦片化编程）：所有的数据搬运、计算、流水线都以 tile 为单位。一个 tile 可以是一块 Shared Memory 里的 16×16 矩阵，也可以是寄存器里的 16×8 fragment。你写的每一条 T.copy、T.gemm 都是在 tile 之间流动数据，编译器负责把 tile 拆解到具体的 Thread 与 Warp。

2.4 编程模型：tile、block、内存层次

2.4.1 @T.prim_func 与 thread block 编程模型

TileLang 用 `@tilelang.jit` 装饰一个 Python 函数，把它编译成 GPU Kernel。函数体内用 `with T.Kernel(...) as (bx, by)` 声明 Grid 与 Block 坐标，整个语法看起来像 CUDA 的“高级版”：

```
import tilelang
import tilelang.language as T

@tilelang.jit(target={"kind": "cuda", "arch": "sm_90a"})
def my_kernel(A, B, C):
    M, N, K = T.const("M, N, K")
    A: T.Tensor((M, K), T.bfloat16)
    B: T.Tensor((K, N), T.bfloat16)
    C: T.Tensor((M, N), T.bfloat16)

    with T.Kernel(T.ceildiv(M, BM), T.ceildiv(N, BN), threads=128) as (by, bx):
        # kernel body
        pass
```

`T.Kernel` 的前两个参数是 Grid 在 y 与 x 方向的 block 数量（注意顺序），`threads=` 指定每个 block 的线程数。`bx, by` 是 block 坐标，类比 CUDA 里的 `blockIdx.x, blockIdx.y`。

直觉 Grid 与 Block 的对应关系是 TileLang 隐藏的“硬件细节”之一。你写 `T.Kernel(Gy, Gx)`，编译器会自动把它映射到 CUDA 的 `<<<(Gx, Gy, 1), (threads, 1, 1)>>>`。你只需要关心“每个 block 算哪一块 C”。

2.4.2 tile 抽象与内存层次

GPU 的存储层次有三个级别，TileLang 把它们显式暴露出来：

层级	TileLang 原语	容量 (H100)	可见性
Global Memory	Tensor / <code>T.Tensor</code>	80 GB HBM	所有 Thread 可见
Shared Memory	<code>T.alloc_shared</code>	228 KB/SM	同一 Block 内 Thread 可见
Register / Local	<code>T.alloc_local</code>	255 KB/SM	Thread 私有
Fragment	<code>T.alloc_fragment</code>	寄存器堆	Warp 协作的 MMA 累加器

Table 11: TileLang 的内存层次抽象

数据流通常是 `Global → Shared → Fragment → Shared → Global`：先把全局数据搬到 Shared Memory（让 Block 内所有 Thread 共享），再加载到 Fragment 调用 Tensor Core 计算，结果写回 Shared Memory 再写回 Global。每一步都对应一条 `T.copy`。

2.5 核心语法速览

TileLang 的原语不多，但每一个都对应一个明确的硬件动作。下面按“分配、搬运、计算、循环”四类介绍。

2.5.1 内存分配

1. `T.alloc_shared(shape, dtype)`: 在 Shared Memory 分配一块 tile, 所有 Thread 可见。常用来缓存从 Global 搬来的 A、B tile。
2. `T.alloc_local(shape, dtype)`: 在寄存器里分配 thread-local 数据, 每个 Thread 独立持有。适合存中间标量或小向量。
3. `T.alloc_fragment(shape, dtype)`: 分配 Tensor Core MMA 的“片段”累加器, 由 Warp 内 32 个 Thread 共同持有。`T.gemm` 的输出通常写到 fragment。

2.5.2 数据搬运

1. `T.copy(src, dst)`: 把数据从 src 搬到 dst。支持 Global $\rightarrow$ Shared、Shared $\rightarrow$ Fragment、Fragment $\rightarrow$ Shared、Shared $\rightarrow$ Global 等组合。编译器会自动选择 `cp.async`、TMA 或 `ldmatrix` 等指令。
2. `T.clear(buf)`: 把 buffer 清零, 常用于累加器初始化。
3. `T.fill(buf, val)`: 把 buffer 填充为标量 val, 比 `T.clear` 更通用。

2.5.3 矩阵乘

1. `T.gemm(A, B, C)`: 在 tile 上做矩阵乘累加 $C += AB$ 。A, B 通常是 Shared Memory tile, C 是 Fragment 累加器。编译器自动选择 `mma.sync`、WGMMMA 等 Tensor Core 指令, 并处理 `ldmatrix` 数据布局。

2.5.4 循环与流水线

1. `T.serial(n)`: 串行循环 n 次, 每步之间没有重叠。
2. `T.Parallel(n)`: 并行循环, 常用于沿 Batch 或 Head 维度的并行。
3. `T.Pipelined(n, num_stages=k)`: 流水线循环, k 级深度, 让下一步的搬运与上一步的计算重叠。

`T.Pipelined` 是 `TileLang` 最有价值的原语之一。手写 CUDA 实现 4 级流水线需要手动管理 4 套 buffer 与 `mbarrier`, `TileLang` 只需 `num_stages=4` 一行, 编译器自动生成 `cp.async` 与 `wgmma.wait_group` 的同步序列。

2.6 最小 GEMM 示例

我们来写一个最小的 GEMM Kernel: $C = AB$, 其中 $A \in \mathbb{R}^{16 \times 16}$, $B \in \mathbb{R}^{16 \times 16}$, 数据类型 BF16。这个例子只有一个 block、一次搬运、一次矩阵乘, 刚好够展示 `TileLang` 的所有核心原语。

```
import tilelang
import tilelang.language as T

@tilelang.jit(target={"kind": "cuda", "arch": "sm_90a"})
def gemm_minimal(A, B):
    M = N = K = 16
    A: T.Tensor((M, K), T.bfloat16)
    B: T.Tensor((K, N), T.bfloat16)
```

```

C = T.empty((M, N), T.bfloat16)

with T.Kernel(1, 1, threads=64) as (bx, by):
    A_shared = T.alloc_shared((M, K), T.bfloat16)
    B_shared = T.alloc_shared((K, N), T.bfloat16)
    C_frag    = T.alloc_fragment((M, N), T.float32)

    T.clear(C_frag)
    T.copy(A, A_shared)
    T.copy(B, B_shared)
    T.gemm(A_shared, B_shared, C_frag)
    T.copy(C_frag, C)

return C

```

逐行批注：第 1 行 `import tilelang` 引入主包，第 2 行 `import tilelang.language as T` 引入 DSL 命名空间，所有原语都挂在 `T` 下。第 4 行 `@tilelang.jit(target={"kind": "cuda", "arch": "sm_90a"})` 装饰器告诉编译器目标是 Hopper 架构 (SM 90a)，会启用 WGMMMA 与 TMA。第 5 行函数签名 `def gemm_minimal(A, B)`，`A`、`B` 是输入 *tensor*。第 6 行 `M = N = K = 16` 是 Python 局部变量，固化问题规模。第 7、8 行用 `T.Tensor` 注解声明 `A`、`B` 的形状与类型，编译器据此推断布局。第 9 行 `C = T.empty(...)` 分配输出 *tensor*，注意 `T.empty` 而非 `T.alloc`，因为 `C` 是 *Kernel* 的输出。

第 11 行 `with T.Kernel(1, 1, threads=64)` 声明 *Grid* 大小 1×1 、每 *block* 64 个 *Thread*，`bx`、`by` 是 *block* 坐标（此处都为 0）。第 12-14 行分配三块片上存储：`A` 与 `B` 的 *Shared Memory tile*，`C` 的 *Fragment* 累加器（用 FP32 累加以保证精度）。第 16 行 `T.clear(C_frag)` 把累加器清零，否则 `T.gemm` 会累加到未初始化的垃圾值上。第 17、18 行 `T.copy` 把 `A`、`B` 从 *Global Memory* 搬到 *Shared Memory*，编译器会自动选择 `cp.async` 或 `TMA`。第 19 行 `T.gemm(A_shared, B_shared, C_frag)` 让 *Tensor Core* 计算 $C_{\text{frag}} += A_{\text{shared}} B_{\text{shared}}$ ，编译器自动 *lowering* 为 `mma.sync` 或 `wgmma.mma_async`。第 20 行 `T.copy(C_frag, C)` 把 *Fragment* 里的 FP32 结果搬回 *Global* 的 BF16 *tensor*，同时完成类型转换与降精度。第 22 行 `return C` 把输出 *tensor* 返回给调用方。

例 假设把上面的 *Kernel* 规模放大到 $M = N = K = 32$ ，并取 *tile* 大小 $B_M = B_N = B_K = 16$ ，那么：

1. *Grid* 大小为 $32/16 \times 32/16 = 2 \times 2 = 4$ 个 *block*。
2. 每个 *block* 负责一个 16×16 的 *C tile*。
3. 沿 *K* 方向需要循环 $32/16 = 2$ 次，每次累加一个 $16 \times 16 \times 16$ 的子矩阵乘。
4. 片上资源：两块 16×16 的 *Shared Memory* (`A`、`B`) 共 $2 \times 256 \times 2 \text{ byte} = 1 \text{ KB}$ ，一块 16×16 的 FP32 *Fragment* 累加器 1 KB，总计 2 KB，远小于 *SMEM* 上限。

如果再增大到 $M = N = K = 4096$ ，*Grid* 变为 $256 \times 256 = 65536$ 个 *block*，每个 *block* 沿 *K* 方向循环 256 次。此时片上资源占用不变（仍是 2 KB），但 *Global Memory*

流量变为 $2 \times 4096^2 \times 2 \text{ byte} = 64 \text{ MB}$ ，访存成为主要瓶颈。这就是为什么需要 T.Pipelined 让搬运与计算重叠。

2.7 与 CUDA/Triton 对比

我们用同一个 Level-2 GEMM ($128 \times 256 \times 64$ tile, 4 级流水线) 来对比三种实现方式：

实现方式	代码量	抽象层级	性能可调性
手写 CUDA	~ 300 行	最低：手写 Swizzle、cp.async、mbarrier、WGMMMA	完全可控，但调试成本高
TileLang	~ 25 行	tile 操作 + 流水线原语	tile size 与 num_stages 可调，其余交给编译器
Triton	~ 30 行	block-level IR，自动选指令	tile size 可调，但 Tensor Core 控制较弱

Table 12: CUDA、TileLang、Triton 三种实现方式对比

TileLang 相比 CUDA 的核心收益是“代码量从几百行压到二十几行”，相比 Triton 的核心收益是“对 Tensor Core 调用与内存层次更可控”。在 Lab3 这种需要精细控制 Fragment 与 Shared Memory 交互的场景下，TileLang 是合适的选择。

2.8 GDN 接口与评测方式

2.9 引言：理解接口和数据流是优化的前提

第一章里我们把 GDN 的数学形式讲透了，但要让它在 GPU 上跑起来，还缺一张“工程地图”：实验框架到底给你哪些张量、要你算什么、又按什么口径评测。本章就是这张地图，把后续所有 TileLang 优化的共同前提一次讲清楚。

我们之所以把“接口”单独成章，是因为这里一个理解偏差就会让后面所有优化功亏一篑。如果你不知道 `g_cumsum` 是 log 空间的前缀和，就会在 kernel 里把它当 `gamma` 直接用；如果你不区分 `initial_state` 是否非零，验证时就会看到莫名其妙的对不上；如果你不清楚 `U / W / S / O` 的边界，shared memory 优化就无从规划。换言之，本章是后续优化的“地基”。

直觉 不妨把 GDN prefill 想象成一条流水线：上游框架已经替你算好 `g_cumsum` 和 `A`，这两份相当于“预处理好的食材”；下游（也就是你的实现）只需要按菜谱做四道菜：`U`、`W`、`S`、`O`。本章的任务就是把这份菜谱每个步骤都拆开讲明白。

读完本章你应该能：完整复述 `gdn_prefill_forward` 的输入输出表，解释 `g_cumsum` 的 log 空间约定，处理 `initial_state` 与 GVA 两种输入变种，把 PyTorch 参考实现按阶段切分，并写出第一个朴素 TileLang baseline 通过正确性验证。

2.10 函数接口详解

实验要求你实现的入口函数是 `gdn_prefill_forward`。固定参数有：chunk size $C = 64$ ，head dimension $d_k = d_v = 128$ 。先看完整的输入输出表：

张量	形状	数据类型	含义
q, k	[B, T, H _q , dk]	BF16	已 L2 归一化的 query / key
v	[B, T, H _v , dv]	BF16	value
g_cumsum	[B, T, H _v]	FP32	chunk 内 log 空间门控前缀和
beta	[B, T, H _v]	FP32	delta rule 的写入强度
A	[B, T, H _v , 64]	BF16	分块 KK^T 下三角矩阵的逆
initial_state	[B, H _v , dk, dv]	FP32	可选初始状态
output	[B, T, H _v , dv]	BF16	prefill 输出
final_state	[B, H _v , dk, dv]	FP32	最终状态

Table 13: gdn_prefill_forward 接口定义

下面我们逐项展开。

q 与 k: query 和 key 已经做过 L2 归一化，形状是 $[B, T, H_q, d_k]$ ，BF16。注意这里 head 数是 H_q ，与下面的 H_v 在 GVA 场景下可能不相等。

v: value 张量，形状 $[B, T, H_v, d_v]$ ，BF16。 H_v 是 value head 数，可能与 H_q 不同。

g_cumsum: 门控前缀和，形状 $[B, T, H_v]$ ，FP32。它是 log 空间的前缀和，也就是说真实使用的衰减因子 $\gamma = \exp(g^{\text{cumsum}})$ 。下文会专门展开。

beta: delta rule 的写入强度，形状 $[B, T, H_v]$ ，FP32，取值通常在 $(0, 1)$ 。

A: 分块 KK^T 下三角矩阵的逆，形状 $[B, T, H_v, 64]$ ，BF16。最后一个维度 64 就是 chunk size C 。它已经把“分块求逆”这一步做完了，你不用在 kernel 里再做矩阵求逆。

虽然 A 名字里带个“逆”，但它真正参与计算时的角色更像一个 *intra-chunk* 的 *gate* 矩阵。你完全可以把它当成一个已经预处理好的 $[C, C]$ 矩阵来用。

initial_state: 可选的初始状态，形状 $[B, H_v, d_k, d_v]$ ，FP32。如果传入 None，表示 $S_0 = 0$ ；否则使用传入的非零状态作为递推起点。

output 与 final_state: 你需要写出的两个张量。output 是 prefill 阶段每个位置的输出，final_state 是把所有 chunk 处理完之后传给后续 decode 的状态。

2.11 门控前缀和的 log 空间约定

g_cumsum 是本章最容易被忽视、又最容易导致结果对不上的一张量。它的命名暗示了它本身不是 gamma，而是 gamma 的 log。

直觉 为什么用 log 空间？因为 gamma 是连乘衰减 $\gamma_1 \gamma_2 \cdots \gamma_t$ ，连乘在浮点数下很容易下溢到 0。取 log 之后变成连加，数值上稳定得多。框架替你算好这个 log 前缀和，你在 kernel 里再 exp 回来。

形式化地说，对 chunk c 内的第 r 个位置 ($r = 1, 2, \dots, C$)，有：

$$\gamma_{c,r} = \exp(g_{c,r}^{\text{cumsum}}), \quad g_{c,r}^{\text{cumsum}} = \sum_{j=1}^r \text{raw}_{g(c,j)}. \quad (2.1)$$

注意三个细节：

1. chunk 内从零起算： $g_{c,0}^{\text{cumsum}} = 0$ ，即 chunk 边界处 gamma 重置为 1。
2. chunk 间的衰减不写进 g_cumsum，而是在状态递推时通过 $\gamma_{c,C}$ 作为整块衰减因子作用到 S_{c-1} 上。
3. exp 必须用 FP32 算，把结果再 cast 回 BF16 与矩阵乘结合。

例 取 $C = 4$ ，假设某 chunk 的 raw log 门为 $[0.0, -0.5, -0.5, 0.0]$ ，则 $g^{\text{cumsum}} = [0.0, -0.5, -1.0, -1.0]$ ，对应 $\gamma = [1.0, 0.607, 0.368, 0.368]$ 。

如果把 g_cumsum 直接当 gamma 用，你会得到 $[0, -0.5, -1, -1]$ 这种负数，与“衰减因子必须在 $(0, 1]$ ”的直觉完全矛盾，从而立刻暴露错误。这也提醒你：写 kernel 时第一件要做的事就是给 g_cumsum 套一层 exp。

2.12 两种输入变种

GDN prefill 在 Lab3 里要正确处理两种输入变种，它们在评测时都会被检查到。

2.12.1 非零 initial_state

最简单的情况是 initial_state 为 None，此时状态从零矩阵 $S_0 = 0$ 开始递推。但评测里也会传入非零 initial_state，模拟“上一段 prefill 留下的状态”或“上一层的跨层状态”。这种情况下，第一个 chunk 的 inter-chunk 部分就不是零，而是 $Q_0 \gamma_0 S_0$ 。

你需要在 kernel 启动时把 initial_state 加载到当前 chunk 的状态寄存器里，作为 $S_{c=0}$ 的初值。final_state 也只有所有 chunk 处理完后才能写出。

2.12.2 GVA: Group-Value Attention

GVA (Group-Value Attention, 分组 value 注意力) 允许 $H_v > H_q$ 。此时 H_v 是 H_q 的整数倍，组大小 $G = \frac{H_v}{H_q}$ 。同一个 query head 对应 G 个 value head，因此 query/key 在 H_q 维度上被复用。

具体规则：对第 h_v 个 value head ($h_v = 0, 1, \dots, H_v - 1$)，其对应的 query/key head 索引为：

$$h_{\text{qk}} = \left\lfloor \frac{h_v}{G} \right\rfloor. \quad (2.2)$$

例 设 $H_v = 8$ ， $H_q = 4$ ，则 $G = 2$ 。value head $h_v = 0, 1$ 都用 $h_{\text{qk}} = 0$ 的 query/key； $h_v = 2, 3$ 都用 $h_{\text{qk}} = 1$ 的 query/key；以此类推。换言之，同一组里的两个 value head 共享同一份 Q 和 K ，但 V 、g_cumsum、beta、A 都按 h_v 独立。

实现上你只需要在 kernel 里多加一行： $h_{\text{qk}} = h_v // G$ ，然后用 h_{qk} 去索引 Q 和 K ，用 h_v 索引其余张量。

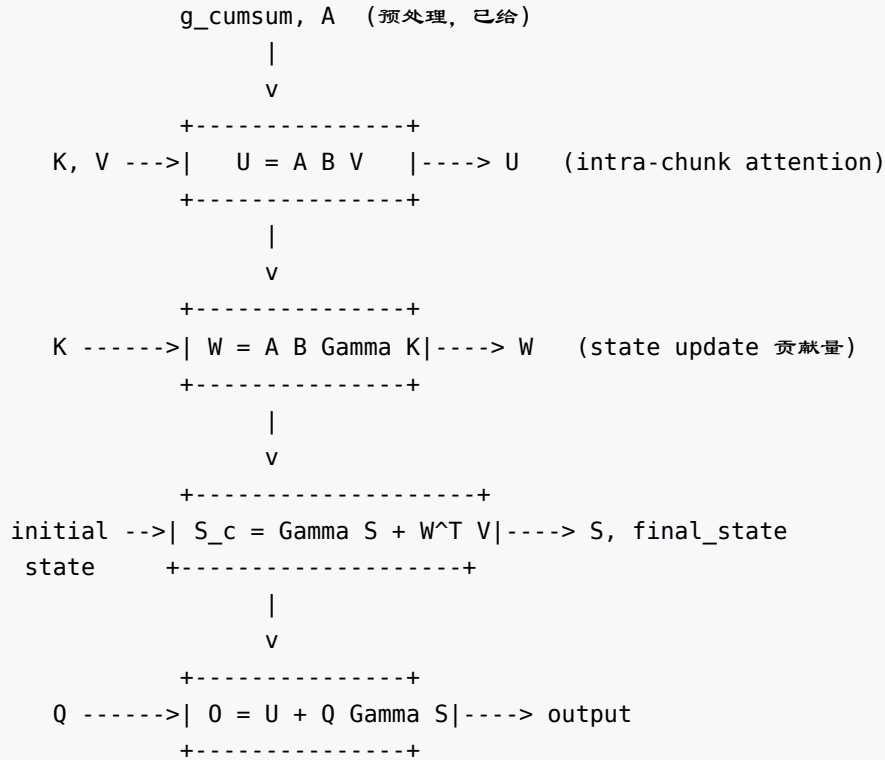
GVA 在工程上是个“易错点”：如果你忘了做 $h_v \rightarrow h_{\text{qk}}$ 的映射，会发现 $H_v = H_q$ 的 case 全对，而 $H_v > H_q$ 的 case 全错。建议在写 baseline 时就专门构造一个 GVA 的测试用例。

2.13 计算阶段分解

把整个 prefill 拆开看，可以这样划分：

1. 预处理阶段（框架已给）： g_cumsum 和 A 。这两个张量不在你的计时范围内，也不用你重新计算。
2. 核心计算阶段（你需要实现）： U 、 W 、 S 、 O 。这是评分依据，下面逐个展开。
3. 后处理阶段（不在计时范围内）：如最终 $final_state$ 的写出、optional 的 normalization 等。

四个核心量的数据依赖关系如下：



下面对每个阶段做一句话解释：

1. U 是 intra-chunk 的注意力输出，只用到本 chunk 的 A 、 V ，输出形状 $[C, d_v]$ 。
2. W 是 state update 的“贡献量”，结合 A 、 γ 、 K ，输出形状 $[C, d_k]$ 。
3. S 是跨 chunk 的状态递推，输入是 S_{c-1} 与本 chunk 的 W 、 V ，输出 S_c 与 $final_state$ 。
4. O 是最终输出，由 intra-chunk 的 U 加上 inter-chunk 的 $Q\gamma S_{c-1}$ 组合而成。

2.14 Nsight Profiling

2.15 引言：优化不能靠猜

你写完了一个 GDN prefill kernel，跑了一下发现比参考实现慢了三倍。你的第一反应可能是“是不是 shared memory 没用好”或者“是不是寄存器太多了”。但 GPU 上有成百上千个因素在影响性能：SM 占用率、访存合并、warp 发散、L2 缓存命中率、kernel launch 开销。靠猜去定位瓶颈，就像闭着眼睛在停车场找车。

性能分析（Profiling）就是睁开眼睛的过程。NVIDIA 提供了两个互补的工具：**Nsight Systems** (nsys) 看整个应用的时间线，告诉你“哪个 kernel 最慢、CPU 和 GPU 之间有没有空等”；**Nsight Compute** (ncu) 看单个 kernel 的内部指标，告诉你“这个 kernel 为什么慢，

是算力不够还是访存不够”。两者配合使用，才能把“慢”从一个模糊的感受变成可定位、可量化的具体瓶颈。

直觉 不妨把 GPU 程序想象成一条流水线工厂。nsys 是工厂的监控摄像头，能俯瞰整条流水线哪台机器在空转、哪台积压；ncu 是工人手里的万用表，能钻进某一台机器看它的电机转速、传送带负载。先用摄像头找到瓶颈工序，再用万用表诊断这台工序的内部问题。

2.16 Profiling 的两个层次

GPU 程序的性能问题可以分成两层：第一层是“整个应用的时间安排有没有问题”，第二层是“某个 kernel 的内部执行有没有问题”。这两层需要不同的工具来回答。

2.16.1 系统级：Nsight Systems

Nsight Systems 关注的是**整个应用的时间线**（application timeline）。它记录的不是某个 kernel 内部的细节，而是“谁在什么时候启动了什么、等了多久、数据在哪里搬动”。它回答的问题包括：

1. 哪个 kernel 占了总执行时间的大头？
2. GPU 有没有大段空闲区间，CPU 在准备数据而 GPU 在干等？
3. kernel launch 是否过于碎片化，成千上万个 small kernel 串行发射？
4. host 和 device 之间的拷贝是否阻塞了计算？
5. CUDA API 调用是否造成了不必要的同步？

典型命令如下：

```
nsys profile -o gdn_baseline ./gdn_test
```

这会在当前目录生成 gdn_baseline.nsys-rep，用 nsys stats 或 Nsight Systems GUI 打开即可查看时间线。

2.16.2 Kernel 级：Nsight Compute

Nsight Compute 关注的是**单个 kernel 的详细性能指标**。它不关心整个时间线，而是钻进一个 kernel 里，回答“这个 kernel 到底卡在算力还是访存”。它报告的关键指标包括 SM occupancy、memory throughput、compute throughput、warp divergence 等。

典型命令如下：

```
ncu --set full -o gdn_kernel ./gdn_test
```

或者只分析某个特定的 kernel：

```
ncu -k gdn_update_kernel --set full ./gdn_test
```

--set full 会收集所有指标（开销较大），调试时常用 --set basic 快速过一遍。

两个工具的使用顺序很重要：先用 nsys 找到最慢的 kernel，再用 ncu 钻进去分析。如果一上来就 ncu --set full 跑全部 kernel，不仅慢，还会淹没在数据里找不到重点。

2.17 Nsight Systems 详解

2.17.1 时间线视图能告诉你什么

打开 nsys 报告后，最重要的视图是**时间线** (timeline)。它把整个程序的执行画成一条条横线：CPU 线程在上，GPU stream 在下，kernel 执行是 GPU 行上的色块，cudaMemcpy 是连接 CPU 和 GPU 的箭头。

你需要关注的几种典型模式：

1. **GPU 空白区**：GPU 行上有大段空白，说明 CPU 在串行准备数据或做同步，GPU 在干等。这是最浪费的情况。
2. **kernel 占比失衡**：如果某个 kernel 的色块占据了大部分时间轴，它就是优化目标。
3. **launch 碎片化**：大量极短的 kernel 色块密密麻麻排列，说明 launch overhead 占比过高，需要 kernel fusion。
4. **同步阻塞**：cudaStreamSynchronize 之后 GPU 才开始干活，说明 host-device 协调有问题。

2.17.2 在 GDN 中怎么看

GDN prefill 的前向过程可以分为四个阶段，对应四组 kernel：

阶段	计算内容	典型瓶颈
U	chunk 内 delta rule 中间量 $u_t = \beta_t k_t^T (v_t - k_t \bar{S}_t)$	小矩阵运算，launch 碎片化
W	$W = K K^T$ 及其下三角求逆 $A = W^{-1}$ 的构造	矩阵乘加三角求解
S	chunk 间状态递推 $S_c = \alpha_c S_{c-1} + \Delta S_c$	访存密集，依赖链长
O	输出 $o_t = q_t S_c$ 的计算	GEMM，算力或访存受限

Table 14: GDN 四阶段 kernel 及典型瓶颈

用 nsys 跑完 baseline 后，你应该在时间线上找到这四组 kernel 的色块，比较它们的总时长。如果发现 U 阶段有成百上千个小 kernel 串行排列，那就是 launch 碎片化的问题，需要 kernel fusion；如果发现 S 阶段的 kernel 执行时间最长，那它就是主要瓶颈，需要进一步用 ncu 分析。

例 假设 nsys 报告显示 baseline 的总执行时间为 1000 μ s，四个阶段的 kernel 时间占比为：U 占 25% (250 μ s)，W 占 15% (150 μ s)，S 占 45% (450 μ s)，O 占 15% (150 μ s)。

从这组数据你能立刻得出两个结论：第一，S 阶段是最大瓶颈，占了近一半时间，应该优先优化；第二，U 阶段占了 25%，如果它的 kernel 数量很多但每个都很短，那 launch overhead 可能是主因，fusion 能大幅压缩这 25%。

2.18 Nsight Compute 详解

2.18.1 关键指标解读

ncu 报告中有几十个指标，但初学者最需要关注以下六个：

指标	含义	理想值	偏低说明
SM Occupancy	活跃 warp / 最大 warp	> 50%	寄存器或 shared memory 过多
Memory Throughput	global memory 带宽利用率	> 50% peak	访存未合并或冗余访问
Compute Throughput	计算单元利用率	> 50% peak	算力不足或依赖链长
Memory Coalescing	线程访问是否合并	接近 100%	访存模式不连续
Register / Thread	每线程寄存器数	视 occupancy 而定	占用高则限制并行度
Warp Divergence	分支发散比例	< 5%	if-else 导致 warp 内分叉

Table 15: ncu 关键指标速查

直觉 SM Occupancy 可以这样理解：一个 SM 最多同时驻留 64 个 warp (V100)，如果你每个线程用了太多寄存器，SM 能容纳的 warp 数就少了，比如只能驻 32 个，那 occupancy 就是 50%。这好比一辆大巴最多坐 60 人，但每个人带了太多行李，实际只能上 30 人，空了一半座位。

2.18.2 在 GDN 中怎么用

拿到 nsys 找到的最慢 kernel 后，用 ncu 对它做 `--set full` 分析。以 S 阶段的状态递推 kernel 为例，你需要回答以下问题：

1. 如果 Memory Throughput 很高但 Compute Throughput 很低，说明它是**访存密集型** (memory-bound)，优化方向是减少 global 访问，用 shared memory 缓存。
2. 如果 Compute Throughput 很高但 Memory Throughput 很低，说明它是**计算密集型** (compute-bound)，优化方向是减少计算量，比如等价数学变换。
3. 如果两者都不高但 occupancy 很低，说明并行度受限，可能是寄存器太多或 block size 不合理。
4. 如果 Memory Coalescing 很低，说明访存模式有问题，线程的访问地址不连续，需要调整数据布局。

ncu 的 `--set full` 会让 kernel 慢 10 到 30 倍，因为它要反复 *replay kernel* 来采集不同指标。调试时先用 `--set basic` 看概况，锁定方向后再用 `--set full` 深入。

2.19 Roofline 模型：判断瓶颈类型

有了 ncu 的 throughput 数据，你已经能初步判断 kernel 是 compute-bound 还是 memory-bound。但更精确的方法是把 kernel 画到 **Roofline**（屋顶线）模型上。这个模型用一个二维图把“算术强度”和“可达性能”的关系画出来，让你一眼看出瓶颈在哪里（参考 HPC101 第 7 章）。

2.19.1 算术强度

算术强度 (Arithmetic Intensity) 定义为每字节内存访问完成的浮点运算数：

$$AI = \frac{\text{FLOP count}}{\text{Byte count}} \quad (\text{FLOP/byte}). \quad (2.3)$$

算术强度高的 kernel 受限于算力，低的 kernel 受限于带宽。这个分界点由 GPU 的平衡点 (ridge point) 决定。

2.19.2 Roofline 公式

V100 有两套计算峰值：CUDA Core 的 FP32 峰值约 15.7 TFLOPS，Tensor Core 的 FP16 峰值约 125 TFLOPS。HBM2 带宽 $B \approx 900 \text{ GB/s}$ 。两个 ridge point 分别为：

$$AI_{\text{FP32}}^* = \frac{15.7 \times 10^{12}}{900 \times 10^9} \approx 17.4 \quad (\text{FLOP/byte}), \quad (2.4)$$

$$AI_{\text{TC}}^* = \frac{125 \times 10^{12}}{900 \times 10^9} \approx 139 \quad (\text{FLOP/byte}). \quad (2.5)$$

Roofline 公式把 kernel 可达性能 P 写为算术强度 A 的函数：

$$P(A) = \min(A \times B, P_{\text{peak}}). \quad (2.6)$$

其中 B 是 HBM2 带宽， P_{peak} 是峰值算力。当 A 小于 ridge point 时， $P = A \times B$ ，kernel 在“带宽屋顶”下，是 memory-bound；当 A 大于 ridge point 时， $P = P_{\text{peak}}$ ，kernel 在“算力屋顶”下，是 compute-bound。

直觉 Roofline 图像一把撑开的伞：左边是斜线（带宽限制，算术强度越高性能越高），上面是水平线（算力限制，性能封顶）。两条线的交点就是 ridge point。你的 kernel 落在斜线上还是水平线上，决定了它的瓶颈类型。

GDN 的 GEMM 操作使用 Tensor Core，所以应该用 TC 峰值 125 TFLOPS 作为屋顶。如果你的 kernel 是 CUDA Core 上的标量运算（比如 delta rule 的逐元素计算），则用 FP32 峰值。

例 假设 S 阶段 kernel 的算术强度为 $A = 50 \text{ FLOP/byte}$ ，实测 occupancy 为 50%。

GDN 的状态递推涉及矩阵乘（使用 Tensor Core），所以参考 TC ridge point 139 FLOP/byte。

由于 $50 < 139$ ，kernel 落在带宽斜线上，是 memory-bound。可达性能 $P = 50 \times 900 = 45000 \text{ GFLOPS} = 45 \text{ TFLOPS}$ ，而 TC 峰值是 125 TFLOPS，利用率只有 $\frac{45}{125} = 36\%$ 。

这说明：即使你把计算逻辑优化到极致，性能上限也卡在带宽上。正确的优化方向是减少访存（用 shared memory 缓存 S ），而不是减少计算。至于 occupancy 50%，说明寄存器或 shared memory 使用偏高，限制了 SM 上同时驻留的 warp 数，这是另一个值得关注的点，但不是当前的主要瓶颈。

2.20 采样与追踪

profiling 有两种采集模式，理解它们的区别能帮你选对工具：

采样 (Sampling) 以固定间隔抽查 GPU 状态，开销低但只有统计信息，适合长时间运行的生产程序。nsys 默认使用采样模式。

追踪 (Tracing) 记录每一个事件 (kernel launch、API 调用、同步等), 开销高但信息完整, 适合调试和优化阶段。nsys 加 `--trace=cuda,nvtx` 后会启用追踪, ncu 的 `--set full` 也是追踪模式。

追踪模式下程序可能慢 2 到 5 倍 (nsys) 甚至 10 到 30 倍 (ncu full), 所以绝对不要用 profiling 数据作为最终性能报告的计时来源。最终计时应该在非 profiling 模式下运行。

2.21 优化闭环

profiling 不是一次性的事, 而是一个**迭代闭环** (iterative loop):

1. **Profile**: 用 nsys 跑一遍, 找到最慢的 kernel。
2. **定位**: 用 ncu 分析该 kernel, 判断是 compute-bound 还是 memory-bound。
3. **假设**: 根据瓶颈类型提出优化方案, 比如“用 shared memory 缓存 S 应该能减少 30% 访存”。
4. **实现**: 编写优化代码。
5. **验证正确性**: 确保输出与参考实现一致。
6. **重新计时**: 在非 profiling 模式下计时, 确认收益。
7. **重新 profile**: 用 ncu 验证瓶颈是否真的被解决, 是否出现了新的瓶颈。
8. **记录**: 把假设、改动、正确性、时间、分析写入迭代日志。
9. **重复**: 回到第一步, 处理下一个瓶颈。

直觉 这个闭环最关键的一点是“每次只改一个变量”。如果你同时改了 shared memory 和 kernel fusion, 性能提升了 30%, 你根本不知道是哪个改动贡献了多少, 甚至可能一个是正收益一个是负收益, 合在一起看像是正的。

2.22 GDN 优化迭代方法论

2.23 引言: 优化是一个迭代闭环

你在 Lab3 的目标是把 GDN prefill forward 的性能从朴素的 baseline 推到接近 FlashQLA 参考实现的水平。这个过程不是“想一个绝妙的优化方案然后一步到位”, 而是“做一步、量一步、想下一步”的迭代闭环。每一步都建立在上一步的 profiling 数据之上, 每一步都要验证正确性和量化收益。

本章把前面几章学过的所有优化技术 (等价数学变换、kernel fusion、shared memory、ping-pong buffer、warp specialization) 串联成一条可执行的优化路径, 并教你如何把整个迭代过程写进 lab3.typ 报告, 让读者看到你的思考过程而不只是最终结果。

直觉 优化就像爬一座未知高度的山。你不知道山顶在哪里, 但每走一步都能看到前方更远一些。profiling 是你的望远镜, 迭代闭环是你的步伐: 看一眼前方 (profile), 选一条路 (假设), 走过去 (实现), 确认没走错 (验证正确性), 回头看走了多远 (计时), 记下来 (日志), 然后继续。

2.24 优化迭代方法论

每一步迭代都遵循以下流程:

1. **Profile**: 用 `nsys` 和 `ncu` 分析当前版本, 找到最大瓶颈。
2. **假设**: 根据瓶颈类型提出优化方案, 预期收益写下来。
3. **实现**: 编写优化代码, 保持接口不变。
4. **验证正确性**: 用参考实现对比 `output` 和 `final_state`, 确保数值对齐。
5. **重新计时**: 在非 `profiling` 模式下运行, 记录新的时间。
6. **记录收益**: 把假设、改动、正确性、时间、分析写入迭代日志。
7. **重复**: 回到 `Profile`, 寻找下一个瓶颈。

每一步只改一个变量。如果你同时改了数学变换和 *kernel fusion*, 性能提升了 30%, 你无法分辨各自的贡献。控制变量法是科学实验的基本原则, 优化也是实验。

2.25 推荐优化路径

优化技术的应用顺序非常重要。有些优化 (如 *warp specialization*) 依赖前面的优化 (如 *shared memory*) 才能发挥作用。以下推荐路径经过实践验证, 每一步都建立在前一步的基础上。

2.25.1 第一步: Baseline

Baseline 的目标是**正确性优先** (*correctness first*)。把 GDN prefill 拆成四个朴素 kernel (U、W、S、O), 每个 kernel 直接翻译数学公式, 不做任何优化。这一步的关键是确保输出与参考实现完全对齐, 为后续优化提供对照基线。

朴素实现的特点是: 每个 kernel 独立发射, 数据在 *global memory* 之间来回搬运。性能会很差, 但正确性有保障。用 `nsys` 跑一遍, 你会看到大量小 kernel 串行排列, GPU 利用率很低。这就是你的起点。

2.25.2 第二步: 等价数学变换

在保证数学等价的前提下, 重新推导公式以减少计算量或改变计算顺序。你可以参考 **FlashQLA** 和 **FLA** (Flash Linear Attention) 的开源实现, 看看它们如何重新排列运算。

一个典型的变换是把逐 token 的递推改写为 *chunk-wise* 的矩阵运算, 用矩阵乘替代标量循环。这不仅减少了循环开销, 还让 Tensor Core 有机会介入。另一种变换是合并门控和 *delta rule* 的计算步骤, 避免重复读写状态 *S*。

这一步不改变 kernel 结构, 只改变数学公式。正确性验证特别重要: 变换前后的输出必须在 BF16 精度下对齐。

2.25.3 第三步: Kernel Fusion

Baseline 的 U 和 W 阶段共享输入数据 (*A*、*B*、 Γ 、*K*), 但分别从 *global memory* 读取。Kernel Fusion 把它们合并为一个 kernel, 数据只读一次, 在寄存器或 *shared memory* 中复用。

类似地, S 阶段的状态递推和 O 阶段的输出计算可以在同一个 *chunk* 内融合: 计算完 S_c 后立刻用它算 o_t , 不需要写回 *global memory* 再读出来。

Fusion 的收益来自减少 *global memory* 访问和 kernel launch 开销。风险是融合后的 kernel 更复杂, 寄存器压力可能上升, *occupancy* 可能下降。

2.25.4 第四步：Shared Memory

把频繁访问的数据缓存到 **shared memory**（共享内存）中。GDN 的主要候选是 K 、 V 、 A 、 S ：它们在 chunk 内被多次读取，每次从 global memory 取都要走 HBM2 带宽。

典型做法：每个 block 负责一个或多个 chunk，把该 chunk 的 K 、 V 、 A 加载到 shared memory，后续计算直接从片上读取。状态 S 也在 shared memory 中维护，chunk 内递推不再触碰 global memory。

这一步对 memory-bound 的 kernel 效果显著，但 shared memory 容量有限（V100 每 SM 96 KB），需要在缓存数据量和 occupancy 之间做取舍。

2.25.5 第五步：ping-pong buffer

ping-pong buffer（双缓冲）利用计算和访存的重叠：当 GPU 在计算 chunk c 的数据时（数据已在 shared memory 中），异步预取 chunk $c + 1$ 的数据到另一个 buffer。计算完后直接切换 buffer，不需要等待数据加载。

这需要两份 shared memory buffer（“ping”和“pong”），空间开销翻倍，但能把访存延迟隐藏在计算时间后面。适合计算和访存时间接近的 kernel：如果计算远大于访存，预取的收益不大；如果访存远大于计算，预取也来不及。

2.25.6 第六步：Warp Specialization

Warp Specialization（warp 分工）把一个 block 内的 warp 分成两组：**producer**（生产者）负责从 global memory 加载数据到 shared memory，**consumer**（消费者）负责从 shared memory 读取数据并执行 Tensor Core 计算。两组 warp 通过异步 barrier 协调。

这种分工让 CUDA Core 的数据加载和 Tensor Core 的矩阵乘并行执行：当 consumer 在算 chunk c 时，producer 已经在加载 chunk $c + 1$ 。与 ping-pong buffer 不同，warp specialization 是在 warp 层面做流水线，粒度更细，能更好地掩盖 CUDA Core 的辅助计算（如 delta rule 的逐元素运算）。

这是最后一步，因为它的实现复杂度最高，且依赖前几步的 shared memory 和 buffer 设计。

2.26 每步的预期收益与风险

步骤	解决什么瓶颈	预期收益	可能的副作用
数学变换	减少冗余计算	5% $\approx$ 10%	数值精度变化
Kernel Fusion	launch 开销, 冗余访存	15% $\approx$ 25%	寄存器压力上升
Shared Memory	global 带宽瓶颈	10% $\approx$ 20%	occupancy 下降
ping-pong	访存延迟	5% $\approx$ 15%	shared memory 占用翻倍
Warp Spec.	CUDA Core 与 TC 不重叠	5% $\approx$ 10%	实现复杂, 调试困难

Table 16: 各步优化的预期收益与风险

预期收益是相对当前版本（不是 *baseline*）的百分比。实际收益取决于你的 *kernel* 具体情况，可能偏高或偏低。如果某步收益远低于预期，说明该步对应的瓶颈不严重，应该优先处理其他步骤。

2.27 迭代日志模板

每一步优化都应该留下记录。以下是一个迭代日志的模板，你可以直接填入 lab3.typ 报告：

迭代	假设	改动	正确性	时间 / 收益
Baseline	-	朴素四 kernel	对齐	1000 μ s / -
1 数学变换	减少冗余 FMA	改写递推为矩阵乘	对齐	950 μ s / -5%
2 Fusion	减少 launch + 访存	U+W 融合, S+O 融合	对齐	760 μ s / -20%
3 Shared Memory	缓存 K/V/A/S	每 block 缓存 chunk	对齐	646 μ s / -15%
4 ping-pong	隐藏访存延迟	双缓冲预取	对齐	581 μ s / -10%
5 Warp Spec.	重叠加载与计算	producer/consumer 分工	对齐	535 μ s / -8%

Table 17: 迭代日志模板（示例数据）

2.28 性能上限分析

优化到一定程度后，你需要问自己：“离理论极限还有多远？”这需要做**性能上限分析** (performance ceiling analysis)。

计算方法如下：

1. 统计 GDN prefill 的理论 FLOP 数 F 。“对于 chunk size $C = 64$ 、 $d_k = d_v = 128$ 、序列长度 T 的输入，主要 FLOP 来自 KK^T ($C^2 d_k$ per chunk)、三角求逆 ($\frac{C^3}{3}$ per chunk)、状态更新 ($C d_k d_v$ per chunk) 和输出计算 ($C d_k d_v$ per chunk)。
2. 查 V100 的峰值算力 P 。“如果用 Tensor Core, $P = 125$ TFLOPS。
3. 理论最短时间 $t_{\min} = \frac{F}{P}$ 。
4. 你的实际时间 t_{actual} 。
5. 利用率 $\eta = \frac{t_{\min}}{t_{\text{actual}}}$ 。

直觉 利用率告诉你“还有多少油可以榨”。如果 $\eta = 50\%$ ，说明你的实现达到了理论峰值的一半，已经很不错了。如果 $\eta = 10\%$ ，说明还有大量优化空间。但如果 $\eta = 90\%$ ，你基本已经触顶了，再花时间优化收益很小。

例 假设 baseline 总时间为 1000 μ s，各步优化的收益（相对当前版本）如下：

1. 等价数学变换：减少 5%， $1000 \times 0.95 = 950 \mu$ s
 2. Kernel Fusion：减少 20%， $950 \times 0.80 = 760 \mu$ s
 3. Shared Memory：减少 15%， $760 \times 0.85 = 646 \mu$ s
 4. ping-pong buffer：减少 10%， $646 \times 0.90 = 581.4 \mu$ s
 5. Warp Specialization：减少 8%， $581.4 \times 0.92 \approx 535 \mu$ s
- 最终从 1000 μ s 降到约 535 μ s，接近减半。注意收益是**乘性叠加**：每步的百分比是相对于上一步优化后的时间，不是相对于 baseline。如果按加性计算 ($5\% + 20\% + 15\% + 10\% + 8\% = 58\%$)，会得到 420 μ s，高估了收益。实际乘性结果是 535 μ s，差距不小。

2.29 与开源实现对比

Lab3 的主要对比基线是 **FlashQLA**，它是 GDN prefill 的参考实现。此外还可以对比 **FLA** (Flash Linear Attention) 和 **FlashInfer** 的相关实现。

对比时需要注意以下几点：

1. 在相同 shape (B 、 T 、 H_q 、 H_v) 下对比，不同 shape 的性能没有可比性。
2. 对比**核心计算时间** (kernel 执行时间)，不含 Python dispatch 和 host-device 拷贝。
3. 注意数据类型：如果参考实现用 FP16 而你用 BF16，Tensor Core 性能可能不同。
4. 如果你的实现比 FlashQLA 慢 20% 以内，已经是很不错的结果；如果慢 2 倍以上，说明还有明显的优化空间。

2.30 填写 lab3.typ 报告

Lab3 的报告不只是贴最终性能数字，更要体现你的**思考过程和分析能力**。以下是你应该在 lab3.typ 中覆盖的内容：

算法与数据依赖分析：解释 GDN 的四阶段 (U/W/S/O) 计算流程，画出数据依赖图，说明哪些阶段可以并行、哪些必须串行。

Baseline 瓶颈分析：用 nsys 和 ncu 的 profiling 数据作为证据，指出 baseline 的主要瓶颈。不要只说“慢”，要给出具体指标，如“S 阶段占总时间 45%，ncu 显示 Memory Throughput 85%，算术强度 50 FLOP/byte，判断为 memory-bound”。

每项优化的设计/实现/收益：对每一项优化，说明你的设计思路（为什么这样改）、实现细节（关键代码片段）和量化收益（优化前后的时间对比）。

最终性能数据：给出每个 case 的 shape、核心计算时间、统计方式（取多少次运行的平均值）、单位 (μs 或 ms)。数据要有量纲和重复次数。

FlashQLA 对比：在相同 shape 下与 FlashQLA 对比，分析差距来源。

尝试但未采用的方案：如果你尝试了某个优化但效果不好或正确性无法对齐，也要记录下来并分析原因。这比只写成功的优化更有价值，因为它展示了你的排除过程。

报告的评分不只看最终性能，更看分析深度。一个从 $1000\ \mu\text{s}$ 优化到 $600\ \mu\text{s}$ 但分析详尽的报告，可能比一个优化到 $400\ \mu\text{s}$ 但只贴数字的报告得分更高。

3 附录: RISC-V 替代路径

3.1 引言：从 x86 到 RISC-V

前面的教程里，我们用 AVX-512 VNNI 和 Intel AMX 在 Sapphire Rapids 上加速了 MoE 前向。这两套指令集都来自 Intel，设计上有一个共同前提：硬件必须提供固定宽度的向量单元 (512 位) 或固定大小的 tile 寄存器 (1 KiB)。程序写死了“一条指令处理 64 个 INT8”，一旦换到向量单元更窄的硬件上就无法运行。

RISC-V (Reduced Instruction Set Computer V) 是另一种思路。它是开源的、模块化的指令集架构，任何人都可以免费使用和扩展。RISC-V 的 **V 扩展** (RVV, RISC-V Vector) 在设计上采用了“向量机”模型：程序员不写死向量长度，而是由运行时的 `vsetvl` 指令根据硬件

实际宽度动态决定一次处理多少元素。同一段代码，在 128 位和 512 位的向量单元上都能正确运行。

更进一步，厂商可以在 RISC-V 的自定义指令区扩展自己的矩阵加速单元。本次 Bonus 使用的 **进迭时空 Muse Pi Pro** 开发板提供了 **SpaceMiT IME** (Integrated Matrix Extension) 矩阵扩展，它复用 RVV 的向量寄存器，把其中的数据理解为小矩阵，用一条 `vmadot` 指令完成 $4 \times 8 \times 4$ 的 INT8 矩阵乘加。

本实验所使用的集群节点为进迭时空 (Spacemit) 为短学期课程建设提供的 *Muse Pi Pro* 开发板。它支持 RVV 1.0 标准和 SpaceMiT IME 扩展，VLEN 为 256 位。

本章先讲 RVV 的编程模型与 Intrinsic 用法，再讲 IME 的矩阵乘指令，最后讨论如何把 Lab2 的 MoE 算子移植到这个平台上。本章代码不能在你的本地 Meteor Lake 或集群的 Sapphire Rapids 上运行，需要在 RISC-V 节点上提交。

3.2 RISC-V 指令集基础

3.2.1 什么是 RISC-V

RISC-V 是 2010 年诞生于 UC Berkeley 的开源指令集架构，由 **RISC-V 国际基金会** 维护。它与 x86、ARM 并列被认为是全球三大主流指令集架构，但与之不同的是，RISC-V 是开放的、非盈利的，任何人都可以免费使用、修改和分发，无需支付许可费用。

RISC-V 的主要特点包括：

1. **开源开放**：免费使用，促进创新与合作
2. **模块化可扩展**：基础指令集加可选扩展（M 乘除法、F 浮点、V 向量等），支持 32/64/128 位
3. **简洁高效**：采用精简指令集原则，指令简单，易于实现，有助于提高能效
4. **全球社区支持**：学术界和产业界广泛参与

3.2.2 RISC 与 CISC 的区别

RISC-V 属于精简指令集 (RISC)，而 x86-64 属于复杂指令集 (CISC)。两者的关键差异在于指令的复杂度和长度。

直觉 RISC 的哲学是“用简单指令组合完成复杂操作”，每条指令只做一件事，但执行快；CISC 则“一条指令干很多事”，但解码复杂。打个比方，RISC 像一把瑞士军刀里的单一工具，每个工具功能单一但高效；CISC 像一个多功能料理机，一个按钮完成多步操作。

以内存访存为例。x86-64 的 `mov` 可以直接完成“基址 + 变址 $\times$ 比例 + 偏移”的复杂寻址，而 RISC-V 需要用多条简单指令拼出来：

```
# x86-64
mov rax, qword ptr [rbx + rcx * 8 + 0x20]

# RISC-V (rax: x1, rbx: x2, rcx: x3)
slli t0, x3, 3          # t0 = (x3 << 3) = x3 * 8
```

```
add    t0, t0, x2          # t0 = x2 + x3 * 8
ld     x1, 0x20(t0)        # x1 = *(t0 + 0x20)
```

另一个区别是指令长度。x86-64 的指令长度可变（1 到 16 字节），解码电路复杂；RISC-V 的指令定长 4 字节（C 扩展中压缩指令为 2 字节），降低了 CPU 设计复杂度。

3.3 RVV 向量扩展

3.3.1 为什么 RVV 不一样

x86-64 的 AVX 要求 CPU 必须包含指定长度的向量单元（128、256 或 512 位），程序写死后只能在匹配的硬件上运行。Intel 曾因能效核心不支持 512 位操作，不得不在 12 代酷睿上禁用 AVX-512，即使同芯片的性能核心支持它。

RVV 和 ARMv9 的 SVE 采用了不同的设计：**硬件厂商可以选择不同的向量单元长度，程序员用同一份代码适配所有长度**。只要编码逻辑正确，相同的程序可以运行在“VLEN”=128 到“VLEN”=512 甚至更宽的硬件上。

RVV 1.0 标准于 2021 年 11 月正式被批准（*Ratified*），后续改动需要保持对该正式版的兼容。本次实验使用的 *Muse Pi Pro* 是目前为数不多支持 RVV 1.0 的开发板。

3.3.2 硬件参数：ELEN 与 VLEN

每个支持向量扩展的硬件线程（hart）有两个硬件参数：

1. **ELEN**：单条指令能处理的最大元素位宽， $ELEN \geq 8$
2. **VLEN**：一个向量寄存器的位宽。RVV 定义 32 个向量寄存器 $v0$ 到 $v31$ ，每个宽度为 VLEN

ELEN 和“VLEN”都必须是 2 的幂，且标准要求 $VLEN \geq ELEN$ 。

例 Muse Pi Pro 拥有 256 位向量运算单元，单元元素最大位宽 64 位，因此：

- ELEN = 64
- VLEN = 256

一个向量寄存器可存 4 个 64-bit 数据、8 个 32-bit 数据、或 32 个 8-bit 数据。

这两个参数由硬件决定，不可在运行时修改。

3.3.3 运行参数：SEW 与 LMUL

硬件给了固定宽度的寄存器，但程序运行时需要控制“一条指令操作什么类型的数据、操作多少个”，这就是“SEW”和“LMUL”的作用：

1. **SEW**（Selected Element Width）：单个向量元素的位宽，如 8、16、32、64
2. **LMUL**（Length Multiplier）：单条指令操作的向量寄存器个数，可取 1、2、4、8 等

当 $LMUL > 1$ 时，指令编码的那个寄存器连同其后 $LMUL - 1$ 个寄存器被视为一个整体，一次性操作更多数据。

例 在 Muse Pi Pro 上 ($VLEN = 256$)，如果 $SEW = 8$ 、 $LMUL = 4$ ，那么一条指令操作 4 个向量寄存器，每个存 32 个 8-bit 整数，共 $4 \times 32 = 128$ 个 INT8 元素。

常见误区：并非“ $LMUL$ ”越大越好。受限于访存带宽和后端运算资源，过大的“ $LMUL$ ”可能无法带来加速，需要实测。另外， $LMUL$ 没有改变单个寄存器的位宽或硬件向量单元宽度，只是改变了单条指令操作的寄存器个数。

3.3.4 Intrinsic 编程

直接手写 RVV 汇编难度较高，编译器提供了 **Intrinsic** 内建函数，让我们像调用函数一样使用底层 RVV 指令，并由编译器完成寄存器分配。使用前需包含头文件：

```
#include <riscv_vector.h>
```

下面是一个完整的 RVV Intrinsic 示例，初始化两个向量寄存器并计算 $c = b + 2.0 \times a$ ：

```
#include <stdio.h>
#include <riscv_vector.h>

float c[8];

int main() {
    size_t vl = __riscv_vsetvmax_e32m1();
    vfloat32m1_t vec_a = __riscv_vfmv_v_f_f32m1(2.0f, vl);
    vfloat32m1_t vec_b = __riscv_vfmv_v_f_f32m1(1.0f, vl);

    vfloat32m1_t vec_c = __riscv_vfmacc_vf_f32m1(vec_b, 2.0f, vec_a, vl);
    __riscv_vse32_v_f32m1(c, vec_c, vl);

    for (int i = 0; i < 8; i++) printf("%f ", c[i]);
    printf("\n");
    return 0;
}
```

编译与运行（需在 RISC-V 节点）：

```
clang test.cpp -o test -march=rv64gcv
srun -N 1 -p riscv ./test
# 输出: 5.000000 5.000000 5.000000 5.000000 5.000000 5.000000 5.000000 5.000000
```

Intrinsic 函数名格式为 `__riscv_<操作>_<形式>_<类型与 LMUL>`。例如

`__riscv_vfmacc_vf_f32m1` 中，`vfmacc` 是浮点乘加，`vf` 表示一个操作数是标量广播，`f32m1` 表示 `float32` 元素、 $LMUL = 1$ 。查询工具推荐社区的 *Intrinsic Viewer*：<https://dzaima.github.io/intrinsics-viewer/#riscv>。

3.3.5 数据类型

RVV Intrinsic 的向量类型以 `v` 开头、`_t` 结尾，中间是元素类型和 LMUL。例如：

类型	元素	LMUL
<code>vint8m1_t</code>	有符号 8-bit	1
<code>vuint8m1_t</code>	无符号 8-bit	1
<code>vint32m2_t</code>	有符号 32-bit	2
<code>vfloating32m1_t</code>	32-bit 浮点	1

Table 18: RVV Intrinsic 常见向量类型

3.3.6 `vsetvl` 系列指令

RVV 适应不同向量长度的关键，是 CPU 内部的 `vl` 寄存器，它记录“接下来的指令要处理多少个元素”。`vl` 的最大值取决于 `VLEN`、`SEW`、`LMUL`。例如 `VLEN = 256`、`SEW = 8`、`LMUL = 1` 时， $vl_{max} = 32$ 。

`vsetvl` 指令根据 `SEW`、`LMUL` 和待处理元素数，结合“`VLEN`”自动更新 `vl`。处理长度为 `N` 的数组时，标准模式是：

```
size_t vl = __riscv_vsetvlmax_e32m1(); // 查询最大 vl
for (size_t i = 0; i < N; i += vl) {
    vl = __riscv_vsetvl_e32m1(N - i); // 动态设置本次 vl
    // 处理 vl 个元素
}
```

直觉 当剩余元素 $N - i$ 大于 vl_{max} 时，`vsetvl` 返回 vl_{max} ，一次处理满；当到达数组末尾不足 vl_{max} 时，`vsetvl` 返回剩余个数，自动忽略多余的 lane。这样无论“`VLEN`”多大、数组多长都能正确处理，这就是“向量机”模型的精髓。

使用 *Intrinsic* 编程时，只需在循环处手动调用 `vsetvl`；编译器会在不同位宽操作之间自动插入 `vsetvl` 切换“`SEW`”和 `LMUL`。手写汇编时则必须在每次切换数据位宽前显式插入 `vsetvl`，否则会触发 *Illegal Instruction*。

3.3.7 常见操作速查

在 *Intrinsic Viewer* 中，Lab2 可能用到的操作分类如下：

分类	子类	用途
Memory	Load / Store	连续或分段访存
Integer	Multiply	INT8/INT32 乘法
Fold	Reduce / Sum	向量元素水平求和（点积尾段）
Conversion	Integer widen	INT8 结果宽化为 INT32
Permutation	Shuffle / Slide	元素位移与重排

Table 19: Lab2 可能用到的 RVV 操作

对于输入 $INT8$ 、结果用 $INT32$ 累加的点积，可使用 *Widening* 操作一次性完成宽化与累加。

3.3.8 Self Check: 内存中的数据有类型吗

学完上面概念后，思考一个问题：内存中的数据，会存储自己的类型吗？

答案是否定的。内存只是无差别的一堆比特位，“数据具有什么含义”取决于你怎样访问和操作它。

直觉 一段 64-bit 数据，可以是 1 个 $int64$ 、2 个 $float$ 、或 8 个 $char$ 。对 CPU 和内存控制器而言，它只是一块 64-bit。真正决定数据类型的是 (SEW, LMUL) 这个二元组配合对应的指令。RVV 的精妙之处在于“数据类型”和“操作”解耦：所有浮点乘法都是 $vfmul.vv$ ，处理多少位、多少个元素由 $vsetvl$ 设置的“SEW”和“LMUL”决定，而不是为每个类型组合定义一条新指令。

3.4 SpaceMiT IME 矩阵扩展

3.4.1 IME 简介

RISC-V 允许厂商在自定义指令区扩展指令集。**SpaceMiT IME** (Integrated Matrix Extension) 是进迭时空提出的矩阵扩展，用于加速低精度矩阵乘法和卷积，宣传中整数运算效率可达 RVV 的 4 倍。

SpaceMiT IME 是厂商扩展，未进入 RISC-V 官方标准。不同厂商（平头哥、SiFive 等）有不同的矩阵扩展。本实验只是借 *SpaceMiT IME* 学习另一种矩阵扩展设计思路，这与具体厂商无关。对设计感兴趣的同学可阅读“RISC-V 矩阵扩展：IME TG Option A-G”等资料。

IME 的“Integrated”含义是它复用 RVV 的向量寄存器，因此不需要像 AMX 那样单独配置 tile 寄存器，直接用向量寄存器即可。

3.4.2 vmadot 指令详解

IME 提供 *vmadot* 系列指令加速矩阵乘法。与 RVV 不同，IME 把寄存器中的数据理解为一个 **小矩阵**，通过专用运算单元完成矩阵乘加，结果仍存回向量寄存器。

进行 8-bit 整数矩阵乘法时，核心指令是：

```
vmadotus vd, vs1, vs2
; us: vs1 无符号, vs2 有符号
; 语义: C += A * B (A, B, C 均为矩阵)
;      vd vs1 vs2
```

在 Muse Pi Pro 上 ($VLEN = 256$)，根据 IME 规范， $M = 4$ 、 $N = 4$ 、 $K = 8$ 。 $vs1$ 和 $vs2$ 中的数据被理解为 $(4, 8)$ 和 $(8, 4)$ 的 8-bit 整数矩阵：

$$A_{4 \times 8} \times B_{8 \times 4} \rightarrow C_{4 \times 4}, \quad C \text{ 为 } INT32. \quad (3.1)$$

直觉 把 vmadot 与 AMX 的 `_tile_dpbssd` 对比：AMX 一条指令做 $16 \times 64 \times 16 = 16384$ 次乘加，需要 1 KiB 的 tile；vmadot 一条指令做 $4 \times 8 \times 4 = 128$ 次乘加，只占用一个 256-bit 向量寄存器。IME 的颗粒度小得多，但它复用了 RVV 寄存器，不需要额外的 tile 配置开销，更轻量。

vmadot 逐行进行内积，结果累加到 32-bit 整数中，最终得到一个 (4, 4) 的 INT32 矩阵。一条指令完成 16 次点积与累加，吞吐量是 RVV 对应指令的 4 倍。

3.4.3 分块矩阵乘法

要利用 vmadot，需要把矩阵分块。对于 $C = A \times B$ ，每次从 A 和 B 中各取一块小矩阵，用 vmadot 乘加后累加到 C 的对应分块。在本例中， $K_{\text{tile}} = 8$ 、 $M_{\text{tile}} = N_{\text{tile}} = 4$ 。

C 矩阵的一块（绿色）= A 的对应分块（蓝色， 4×8 ） $\times$ B 的对应分块（黄色， 8×4 ）的累加和。

沿 K 方向每 8 列切出 A 的一块，沿 K 方向每 8 行切出 B 的一块，多次 vmadot 累加得到完整 C 块。

$K_{\text{tile}} = 8$, $M_{\text{tile}} = N_{\text{tile}} = 4$ 。

Listing 4: 分块外积矩阵乘法示意

为了“加载一小块分块矩阵到寄存器”，可能需要 *Strided Load*（按跨步加载）。注意加载和操作数据时可设置不同的“SEW”与 LMUL：例如用 $\text{SEW} = 8$ 的 *Strided Load* 取数据，再用 IME 指令做矩阵乘。但这只是参考，*Strided* 访存未必最优，重排布局后连续访问可能更快。

3.5 在 Lab2 中应用 RISC-V

3.5.1 从 x86 到 RISC-V 的迁移

Bonus 任务延续主线：输入、权重、路由、量化和输出都不变，只把 moe_forward_optimized 的目标平台换成 RISC-V。移植时需要做以下对应：

功能	x86 路径	RISC-V 对应
Router FP32 点积	AVX-512 FMA <code>_mm512_fmadd_ps</code>	RVV <code>__riscv_vfmacc</code>
INT8 点积	VNNI <code>_mm512_dpbussd_epi32</code>	RVV Widening 或 IME vmadot
批量 INT8 GEMM	AMX <code>_tile_dpbssd</code>	IME vmadot 分块
水平求和	<code>_mm512_reduce_add_epi32</code>	RVV Fold / Reduce
权重预打包	preprocess 中重排	同样适用，布局按 IME 分块调整

Table 20: x86 与 RISC-V 优化路径对照

3.5.2 IME 与 AMX 的关键差异

移植时最需要重新思考的是矩阵乘路径。IME 与 AMX 在几个方面差异显著：

1. **有符号性**：AMX 的 `tdpbssd` 直接做 `signed × signed INT8`；`vmadot` 的 `us` 后缀表示无符号 `×` 有符号。若 `W8A8` 激活和权重都是有符号 `INT8`，需要像 `VNNI` 那样用偏移技巧，把其中一个转为无符号，再做主点积并减去修正项。
2. **分块形状**：AMX tile 是 16×16 输出；IME 是 4×4 输出，颗粒度小 16 倍。单 token 的矩阵向量乘用 IME 已经足够，但批量场景需要更多循环层数。
3. **配置开销**：AMX 需要 `_tile_loadconfig` 配置 `palette`；IME 复用 RVV 寄存器，无额外配置，但要注意 `vsetvl` 的 `SEW/LMUL` 必须与 `vmadot` 期望的数据布局匹配。
4. **寄存器压力**：AMX 有 8 个独立 tile；IME 共用 RVV 的 32 个向量寄存器，矩阵乘与辅助计算（`SwiGLU`、量化）要一起分配。

3.5.3 适配策略

参考前面 x86 的两级内核设计，RISC-V 上也可保留 `VNNI` 风格的小批量路径与 IME 的批量路径：

1. **单 token 路径**：用 RVV Widening 做 `INT8 × INT8 → INT32` 点积。`VLEN = 256`、`SEW = 8`、`LMUL = 1` 时一条指令处理 32 个元素，配合偏移技巧处理有符号性。`S1`、`S2` 走这条路径。
2. **批量路径**：按专家分组后，对 M 个 token 的 $M \times K$ 量化激活与 $K \times N$ 权重做 IME 分块乘加。 $K_{\text{tile}} = 8$ 要求权重按 8 列为一组打包，激活按 8 行为一组加载。`S3`、`S4` 的共享专家与较大路由专家组走这条路径。
3. **动态分派**：与 x86 的 `amx_threshold` 类似，设置 `ime_threshold`，专家组大小低于阈值时回退到 RVV 路径，避免小矩阵的 tile 利用率过低。

由于 *Muse Pi Pro* 的“`VLEN`”只有 256，且 IME 单指令只做 4×4 输出，绝对性能远低于 *Sapphire Rapids* 的 AMX。*Bonus* 的评分不应只看绝对加速比，而应关注“是否正确利用了 RVV 与 IME 的特性”以及“相对标量基线的提升”。

3.6 编译与运行

在 RISC-V 节点上编译需要指定 `rv64gcv` 架构以启用 V 扩展，IME 扩展需要进迭时空工具链支持：

```
clang -O2 -march=rv64gcv -o test test.cpp
srun -N 1 -p riscv ./test
```

提交方式与主线任务的入口仍在完善，开放后会更新文档并通知。

讲义基于 HPC101 2025 Lab2 与 Lab3 实验内容编写